MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

James Gabriel P. Mabagos
Bachelor of Science in Computer Science

Mark Lawrence M. Quibot
Bachelor of Science in Computer Science

Senior Thesis submitted to the faculty of the
Department of Computer Science
College of Computer Studies, Ateneo de Naga University
in partial fulfillment of the requirements for their respective
Bachelor of Science degrees

Project Advisor: Marianne A. Tolentino
Estrella H. Montealegre
Michelle Elija B. Santos
Ian Peter L. Lastimosa

September 5 2025
Naga City, Philippines

Keywords: Imperfect Information, DQN, Multi-Agent

The Senior Project entitled

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

developed by

James Gabriel P. Mabagos

Bachelor of Science in Computer Science

Mark Lawrence M. Quibot

Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees has been rigorously examined and recommended for approval and acceptance.

Estrella H. Montealegre

Panel Member

Date signed: ____________________

Michelle Elija B. Santos

Panel Member

Date signed: ____________________

Ian Peter L. Lastimosa

Panel Member

Date signed: ____________________

Marianne A. Tolentino

Project Advisor

Date signed: ____________________

The Senior Project entitled

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

developed by

James Gabriel P. Mabagos
Bachelor of Science in Computer Science

Mark Lawrence M. Quibot
Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees is hereby approved and accepted by the Department of Computer Science, College of Computer Studies, Ateneo de Naga University.

Lowie Vincent S. Bisana MIT
Chair, Department of Computer Science
Date signed: ________________

Joshua C. Martinez MIT
Dean, College of Computer Studies
Date signed: ________________

Declaration of Original Work

We declare that the Senior Project entitled

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

which we submitted to the faculty of the

Department of Computer Science, Ateneo de Naga University

is our own work. To the best of our knowledge, it does not contain materials published or written by another person, except where due citation and acknowledgement is made in our senior project documentation. The contributions of other people whom we worked with to complete this senior project are explicitly cited and acknowledged in our senior project documentation.

We also declare that the intellectual content of this senior project is the product of our own work. We conceptualized, designed, encoded, and debugged the source code of the core programs in our senior project. The source code of third party APIs and library functions used in our program are explicitly cited and acknowledged in our senior project documentation. Also duly acknowledged are the assistance of others in minor details of editing and reproduction of the documentation.

In our honor, we declare that we did not pass off as our own the work done by another person. We are the only persons who encoded the source code of our software. We understand that we may get a failing mark if the source code of our program is in fact the work of another person.

James Gabriel P. Mabagos
4 - Bachelor of Science in Computer Science

Mark Lawrence M. Quibot
4 - Bachelor of Science in Computer Science

This declaration is witnessed by:

Marianne A. Tolentino
Project Advisor

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

by

James Gabriel P. Mabagos and Mark Lawrence M. Quibot

Project Advisor: Marianne A. Tolentino

Department of Computer Science

ABSTRACT

This research presents MARQ, a Multi-Agent Rainbow Deep Q-Networks, designed to master imperfect information games through the board game Stratego. MARQ leverages a Deep Recurrent Q-Network (DRQN) architecture and league-based self-play training to address the large state space of Stratego and hidden information challenges. The architecture integrates an Attention as Recurrent Neural Network (AAREN) module that produces learned embeddings from observed opponent behavior, enabling implicit piece identity inference through end-to-end training. The system employs Distributional Reinforcement Learning with Noisy Networks to balance strategic unpredictability and stability. The methodology involves developing a game environment and training the AI agent through multi-channel state representations, incorporating AAREN-generated embeddings for hidden information modeling and weighted piece values computed through systematic power analysis to optimize decision making under uncertainty. A structured participant study introduces enthusiasts without prior Stratego experience to create a synchronized learning environment, combining quantitative metrics including win rate percentage, average game length, and decision-making speed, with qualitative usability assessments across performance, reliability, and applicability dimensions. Through this study MARQ aims to deliver strategic decisions comparable to human opponents.

ACKNOWLEDGEMENTS

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

Chapter 1

Introduction

1.1 Project Context

Games have a history of being a metric for how Artificial Intelligence (AI) progresses and performs in the current time [49]. Many real-world problems, from military tactics to multiplayer games, require intelligent decision-making in environments where all information is not available. A common example is a feature found in many multiplayer games known as the "fog of war" [64], where the game environment is not fully revealed, which makes it an example of an imperfect information game [33]. Imperfect information games create a scenario where players develop unconventional strategies without having full knowledge of the game's environmental state [64]. Solving these kinds of problems and determining the optimal strategy remains a significant challenge for Artificial Intelligence [50].

This study focuses on mastering imperfect information games through Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN). In MARL, multiple agents learn and adapt in the same environment, making each decision based on its own observations and rewards [19, 15]. With DQN, the agents can utilize deep neural networks to approximate the Q-function that maps state-action pairs for the expected reward. This study introduces the MARQ (Multi-Agent Rainbow Deep Q-Networks) framework, which extends the basic DQN architecture with Rainbow enhancements including distributional value estimation and noisy exploration networks [26]. The framework integrates an AAREN (Attention as Recurrent Neural Network) module [23] for processing extended observation sequences and producing learned embeddings that enable implicit inference of hidden opponent pieces through end-to-end training. League-based self-play training [53] ensures that agents develop robust strategies by competing against diverse historical opponents. This approach is suited well for Stratego, where agents are trained to compete against each other, developing strategies through repeated experience while reasoning under uncertainty about the opponent's setup and the environment's hidden state.

1.2 Purpose and Description

This research aims to evaluate whether Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN) can be used to develop intelligent agents capable of mastering imperfect information games. By focusing on the limited information available to players in environments such as the board game Stratego, this study seeks to address the challenge of making decisions under uncertainty while incorporating strategic planning. Although existing tools for Stratego provide some analytical support, they are often limited in fostering deeper reasoning about the game's tactical complexities.

To advance beyond these limitations, this research seeks to provide an algorithmic foundation for future analytical systems while also emphasizing the development of critical thinking in solving imperfect information games. By modeling adaptive strategies and reasoning under uncertainty, the study highlights how artificial intelligence can mirror and enhance human-like logical reasoning, offering valuable insights for both game analysis and broader decision-making domains.

1.2.1 Research Questions

This study is guided by three main research questions:

1. How does the MARQ framework perform in terms of convergence speed and win-rate against heuristic baselines in a partially observable environment?

2. Does the MARQ framework demonstrate statistically significant superiority over standard DQN and rule-based agents across varying opponent difficulty tiers?

3. What is the impact of integrating the AAREN module on the agent's ability to infer hidden opponent piece configurations compared to approaches without explicit belief state modeling?

1.3 Objectives

The main objective of this study is to develop and evaluate Multi-Agent Reinforcement Learning with Deep Q-Networks for learning appropriate strategies in an imperfect information game, where the agents operate under hidden state variables to improve decision-making, adaptability, and strategic reasoning against both human and AI opponents. This can be achieved through the following:

- Finding ideal algorithms to be implemented in perfecting Stratego.
- Design and implement a game environment for Stratego
- Build and implement a training model using Multi-Agent Reinforcement Learning with Deep Q-Networks and AAREN.
- Train and optimize the AI agent to respond to varying game states, hidden information, and opposing strategies.

1.4 Scope and Limitation

The study is limited to the development of an AI agent for imperfect information games, Stratego being the test environment. The scope includes the design and implementation of Stratego as a game environment, the development of a training framework using Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN) using AAREN, and the training and optimization of the AI agent to adapt to differences in game states. The evaluation of the agent's performance would primarily be on Stratego's environment. The trainings are limited to the personal devices of the researchers and would not rely on cloud-based computing resources. Since the AI is going to learn alongside the community the player and AI head-to-head will be analyzed mostly qualitative. The goal will be achieving the same human critical thinking in the model.

1.5 Significance of the Study

The development of the MARQ framework addresses limitations in current imperfect information game AI, where existing systems either require large computational resources or employ model-free approaches that limit strategic depth. The MARQ framework's significance lies in its implicit opponent modeling through AAREN-generated embeddings, and hybrid DQN architecture that balances strategic depth with computational tractability. The synchronized learning evaluation framework provides insights into AI adaptability against improving human opponents, with implications extending beyond gaming to domains characterized by incomplete information and strategic interaction, including cybersecurity, financial trading, and strategic planning.

1.6 Definition of Terms

AAREN (Attention as Recurrent Neural Network)
A neural architecture that combines the parallel training efficiency of Transformers with the sequential processing of RNNs. In the MARQ framework, AAREN processes observation sequences to produce learned embeddings that encode implicit piece identity inference.

Counterfactual Regret Minimization (CFR)
An iterative algorithm that approximates Nash equilibrium by minimizing regret, widely applied in imperfect information games such as Poker and Stratego.

Deep Q-Networks (DQN)
A reinforcement learning algorithm that combines Q-learning with deep neural networks, enabling agents to approximate optimal decision-making in complex environments.

Distributional Reinforcement Learning
An approach that models the full distribution of returns rather than just the expected value. The MARQ framework uses C51 (Categorical DQN) with 51 atoms to capture uncertainty in value estimates.

Fog of War
A game mechanic used in strategy games that conceals parts of the map or opponent actions, requiring players to utilize prediction and information-gathering strategies to reduce uncertainty.

Imperfect Information Games

Games where players have incomplete knowledge of the current game state, requiring them to make strategic decisions under uncertainty (e.g., Stratego, Poker, Multiplayer Online Battle Arenas).

Information Set Monte Carlo Tree Search (IS-MCTS)

A search algorithm for imperfect information games that samples consistent game states from belief distributions and performs tree search on information sets rather than individual states.

League Training

A self-play training paradigm where agents compete against a diverse pool of historical opponents, preventing strategy cycling and ensuring robust generalization across different playing styles.

Multi-Agent Reinforcement Learning (MARL)

An extension of reinforcement learning in which multiple agents interact within the same environment, learning coordinated or competitive strategies through simultaneous training.

Nash Equilibrium

A game-theoretic concept where no player can gain an advantage by unilaterally changing their strategy, often used as a benchmark for decision-making in imperfect information games.

Perfect Information Games

Games in which all players have complete knowledge of the current game state and available moves, such as Chess and Go.

AAREN Embeddings

Embedding vectors produced by the AAREN module from observed opponent action sequences.

Rainbow DQN

An enhanced DQN architecture that combines multiple improvements including distributional RL, noisy networks for exploration, and dueling network architecture for improved stability and sample efficiency.

Recurrent Neural Network (RNN)

A type of neural network designed to process sequential data by maintaining memory of previous inputs, useful in imperfect information games for recalling past moves or revealed information.

Reinforcement Learning (RL)

A machine learning paradigm where agents learn optimal strategies through trial-and-error interactions with the environment, guided by rewards and penalties.

Stratego

A two-player strategy board game characterized by hidden information and asymmetric piece abilities, where the objective is to capture the opponent's flag or render them immobile.

Chapter 2

Review of Related Literature

2.1 Artificial Intelligence in Imperfect Information Games

Games can be classified as an imperfect information game by having incomplete information about the current game state for both players [49, 19]. An example would be in MOBAs (Multiplayer Online Battle Arena), the fog of war mechanic hides portions of the map, including objectives and enemy positions, from both teams. By placing a tool, such as a ward in League of Legends or an observer in Dota 2, the player gains a temporary vision in the map for strategic planning [17]. This feature engages players to create strategic reasoning under certainty, utilizing prediction, coordination, and resource management to gain informational advantages over opponents. Games have a history of being a metric on how AI progresses and performance in the current time [49, 27]. The case is the same for both perfect information and imperfect information games [49]. For perfect information games, like chess, they primarily use Alpha-Beta pruning, a search algorithm that uses trees to explore possible chess moves and prunes the branches that will not affect the final decision [41]. In imperfect information games, a search algorithm like Alpha-Beta Pruning would be difficult to implement because the algorithm is made for perfect information games [47]. A popular algorithm that is used in imperfect information games would be Monte Carlo Tree Search with Reinforcement Learning because MCTS has integration with neural network systems like AlphaZero [43, 11]. In terms of reinforcement learning, the training focuses on trial and error to provide the best possible decision [67]. Continuing development of AI algorithms in imperfect information games advances our understanding of decision-making under uncertainty and promotes strategic planning.

2.1.1 Student of Games

A unified learning algorithm that can support both perfect information and imperfect information is Student of Games. Combining self-play learning, strategic search, and principles from game theory [49]. SoG achieved strong performance in Chess and Go in perfect information games and in imperfect information games like heads-up no-limit Texas hold 'em poker, and defeated the state-of-the-art agent in Scotland Yard. When comparing to MARL, SoG supports and has been tested in both game types and uses game theoretic reasoning, which computes or approximates the Nash equilibria. MARL with DQN uses trial and error learning and self-play learning, which relies on the exploration of the environment. MARL has also been tested in Starcraft II, which is a real-time strategy video game, unlike SoG, which has not been tested in video games [29].

2.1.2 Deep Q-Networks and Dueling Architectures

Deep Q-networks (DQN) and their variants have become a standard framework for decision-making in complex environments with high-dimensional observations. Hessel et al. (2021) combined several independent improvements to DQN—such as double Q-learning, prioritized replay, dueling networks, and noisy nets—into a unified agent called Rainbow, demonstrating state-of-the-art performance on competitive benchmarks [26]. This demonstrates that carefully integrating architectural and algorithmic improvements can substantially improve both data efficiency and final performance in value-based deep RL.

A key architectural component of Rainbow is the dueling network architecture, proposed by Wang et al. (2016) and detailed in modern surveys [26, 54]. This design splits the Q-function into a state-value stream and an advantage stream. This split allows the network to generalize across actions and improves policy evaluation, particularly in environments with many similar-valued actions. Our MARQ model builds on this idea by feeding the shared ResNet-based representation into dueling value and advantage streams, which is particularly beneficial in board games where many positions have similar strategic value.

2.1.3 Multi-Agent Reinforcement Learning

Reinforcement Learning can solve complex problems through trial and error, by learning from the environment to be able to provide optimal decisions [19]. This can be the case for Single-Agent Reinforcement Learning(SARL). SARL has its limits in solving many real-world problems. Multi-Agent Reinforcement Learning(MARL) was introduced as a solution for the challenges of SARL, by having multiple agents in an environment that interact with each other, the decisions they could make would be optimized [19, 15]. Optimized, meaning that these agents go through trial and error together to make the best decision given the reward. As for the implementation of this in Stratego, two agents would compete with each other to know the optimal decisions given the environment, which has imperfect information and opposing strategies. As MARL has its advantages, it also has challenges. The first would be Non-Stationarity, and the second is scalability, as for non-stationarity, each agent learns simultaneously, meaning the environment constantly changes [15]. For scalability, as more agents are being used, the computational power rises, making it expensive [37, 15]. Recent work on Ataraxos demonstrates that these challenges can be addressed through careful regularization: by constraining policy updates toward both exploratory (uniform) and stable (target network) anchors with time-varying coefficients, agents can achieve superhuman performance in complex imperfect-information games while maintaining training stability [52].

2.1.4 Counterfactual Regret Minimization

Counterfactual Regret Minimization(CFR) is an iterative algorithm that approximates the Nash equilibria commonly used on imperfect information games like poker and Stratego [37, 61]. The Nash equilibria are what CFR approximates to make the best possible decision with minimal regret. Regret in this scenario is the what-ifs of the algorithm if the algorithm chooses the other decision. For CFR to make decisions, it repeatedly minimizes regret to approximate the Nash equilibria [37]. Other works have made variations of the base CFR, showing improved convergence rate, but require a significant amount of effort by researchers. A framework is proposed named AutoCFR, instead of relying on manually crafted algorithms, it automatically designs new CFR variants using an outer loop to search over algorithm designs and an inner loop to test them on equilibrium finding tasks [60].

2.1.5 Opponent Modeling in Imperfect-Information Games

In imperfect-information games, an agent's optimal strategy depends not only on the environment but also on the behavior of the opponent. Classical opponent modeling often builds models from observed action frequencies, and computes best responses in a game-theoretic framework. Li et al. (2024) formalize opponent-model search in games with incomplete information, providing algorithms to compute optimal or robust strategies given various forms of opponent knowledge [36].

In the deep RL setting, modern approaches encode observations of opponents directly into the network trunk, jointly learning a policy and an opponent model without explicitly predicting actions [54]. This shows that learning implicit opponent representations from interaction traces can be more effective than handcrafted models, especially in high-dimensional state spaces. Our AAREN module follows this tradition of implicit opponent modeling but is specialized to board games with hidden piece identities. Instead of maintaining explicit probability distributions over opponent piece types, AAREN aggregates observed opponent action sequences into dense embedding vectors. These vectors are then concatenated as additional input channels to the Q-network, allowing the agent to reason about the opponent's likely pieces and strategy in a learned representation space.

2.2 RNN

Recurrent Neural Network is a type of Neural Network designed to process sequential data. Unlike common Neural Networks, RNNs have a memory because the information passed to the next step is based on the memory from the previous step. RNN can be applied to imperfect information games such as Stratego because of its memory. By using the memory to remember the pieces that have been revealed, the decision could be optimized [43].

2.2.1 History Aggregation and State Representation Learning

Learning from histories is a central challenge in partially observable and non-Markovian environments. Wang et al. (2026) show how to systematically construct non-Markovian decision processes by aggregating interaction histories into state representations, providing an effective way to add temporal dependencies into RL [56]. By explicitly modeling history via learned aggregators, an agent can better handle complex temporal structures.

More broadly, representation learning for RL has explored the use of embeddings to generalize across states and goals. Echchahed and Castro (2025) highlight in their survey that separating dynamics from rewards allows policies to be transferred across tasks through learned representations [21]. Our AAREN embeddings can be viewed as a form of history-dependent state representation that encodes the opponent's action history into a compact vector to augment the board state. This approach is tailored to the specific structure of imperfect-information board games where opponent behavior reveals hidden state. By learning these embeddings jointly with the Q-network, we avoid the need to handcraft belief distributions and instead let the network infer implicit piece identities from observed behavior.

2.2.2 LSTM

Long Short-Term Memory is a popular RNN variant that has the capabilities of RNN but also processes data with long-term dependencies [2]. LSTM mitigates the vanishing gradient problem that RNN faces. The vanishing gradient problem is basically as in the name, vanishing gradient, as color in a gradient, the message in this case starts strong and gradually fades away when it passes through each layer, making the learning weak or close to zero. LSTM can be used to solve imperfect information games because of its capability to process data with long-term dependencies while having the memory of RNN.

2.2.3 Aaren

Aaren, known as Attention as a recurrent neural network, combines the parallel training efficiency of Transformers with the streaming update efficiency of RNNs [23]. Aaren, like transformers, can be trained in parallel as well as RNN, can process sequential data with memory. In the study, Aaren was tested in event forecasting, time series forecasting, and classifications. Aaren achieved similar accuracy to Transformers but was more time and memory-efficient.

2.2.4 Residual Networks and Self-Attention in Board Games

Residual networks (ResNets) have become a common backbone in modern board-game RL systems. AlphaZero-style agents for Go, chess, and Stratego use a deep tower of residual convolutional blocks as the network trunk, enabling stable training of deep networks that can capture spatial dependencies on the board [11]. Modern architectures often mix these residual blocks with self-attention layers to bridge local and global knowledge. This hybrid architecture significantly improves playing strength in board games by better handling long-range patterns [5].

Because of these results, our architecture adopts a ResNet backbone with spatial self-attention. This allow each board position to attend to all other positions, capturing long-range piece relationships that are difficult to represent with purely convolutional architectures. We extend this line of work by combining self-attention with a dueling Q-network and a learned opponent embedding, addressing the additional challenges of imperfect information and opponent modeling in Stratego.

2.3 Stratego

The origin of Stratego can be traced to a traditional board game called Jungle, where instead of human pieces instead it used animals. In the jungle, the pieces are not hidden, making it a perfect information game. Stratego is a two-player board game where players each have a 40-piece army. In the tabletop version of Stratego, one player is assigned to the blue army and the other player is assigned to the red army. Each army has a flag assigned to it, which cannot be moved when the game starts. There are a total of 80 troop pieces, while the total number of variations is 12. The pieces rank range from 1 to 10, and a piece that's not numbered is a bomb and the flag. The player also has time to move pieces before the game starts to make use of the fog of war and create creativity and optimal positions on the board. As the pieces have ranks, the higher the number, the higher the power. There is an instance where the lower-ranked can defeat a higher-ranked ranked, which is the piece that has a rank of one, named the spy, can defeat the strongest piece that has a rank of ten, named the marshal, if the spy attacks first. As for the bombs, almost all pieces are vulnerable except the miner, the only one who can defuse the bomb. And the pieces that cannot be moved are the bomb and the flag. Another piece that has a special ability is the scout; it can move either horizontally or vertically, not just one square. If both pieces have the same number, and the piece attacks, both of them would be eliminated. There are multiple win conditions in Stratego. The obvious one would be capturing the flag, another one would be that the opponents have no mobile pieces, another would be that there are no miners that can defuse the bomb to capture the flag, and lastly, the opposing player had the time run out.

2.3.1 Deep Reinforcement Learning in Stratego

Stratego is a popular imperfect-information board game where players must reason about hidden piece identities over long time horizons. Pérolat et al. (2022) demonstrated with DeepNash that a model-free multiagent RL approach can reach human expert level in Stratego, using deep convolutional networks trained with game-theoretic regularization [45]. Their architecture relies heavily on ResNet-style networks to process board features under imperfect information. More recent work has further improved these baselines by combining self-play reinforcement learning with test-time search, achieving superhuman performance [52].

Our MARQ framework is closely related to these systems in its use of a deep ResNet backbone, but we explicitly incorporate opponent modeling via AAREN embeddings. By basing the agent on the opponent's observed actions, we allow the agent to adapt to specific opponents and exploit behavioral patterns. This adds to the equilibrium-style play of prior models with learned opponent representations.

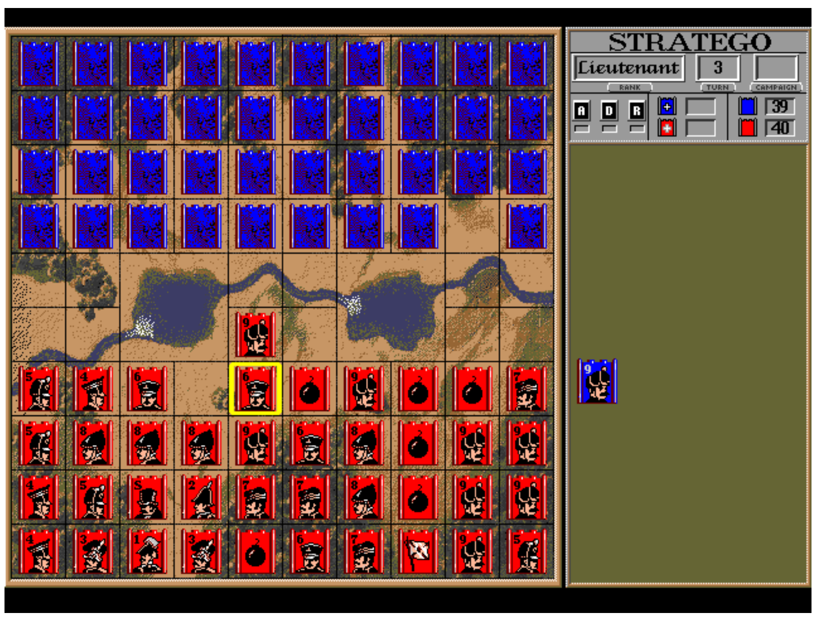


Figure 2.1: Stratego Game board.

2.4 Board Games in Community Building

Modern board games are becoming more widespread, expanding their market share each year. During COVID-19 lockdowns, board game sales increased because people were stuck at home with family and had free time. Also, playing board games during that time helped with stress, isolation, and anxiety. Certain games like Pandemic, a cooperative strategy board game where players work together to stop global disease outbreaks by discovering cures before time runs out, became more popular due to their reflection of the real-world scenario. Board game communities differ between the West and the East [24]. From the east, board games are considered social activities, while in the west, board games grew as hobbies, communities focusing on the gameplay and mechanics. In Hong Kong, there are two distinct board game communities named BGHK and Oasis. BGHK consists of Cantonese-speaking individuals who treat board games as social bonding. They even event sessions for "party game days" and "welcome newbies," emphasizing making friends rather than the game itself. While the Oasis group focuses on the hobby, playing newly released and complex games. Attracting players with the mechanics and novelty rather than just socializing. A study has been made to use board games as an answer to the problem of difficulty in creating a sense of "togetherness or kinship," and board games have advantages for improving the cognitive function of older people [3]. In the study, the Tresna Werdha Budi Pertiwi Social Home has a problem of having a sense of family among residents. By playing together, the elderly had consistent interaction with each other, reducing the feeling of loneliness and detachment. Board games not only provide entertainment but also memory stimulation and improve social bonds. Board games provide a face-to-face interaction requiring people to sit together at a table, unlike in video games, where players are often separate from each other [44]. Every shared experience creates a unique microenvironment where players enter a separate reality governed by the game's rules. In the space, the players share emotions and strategy. Sharing these emotions helps strengthen friendships, family bonds, and trust. Many board game players first connect through gaming with friends and family, which can branch out to dedicated groups or conventions, and expand their social circle.

2.5 Other Applications

An extension of MARL with DQN was demonstrated in robotics. In these environments, robots are trained to make decisions based on rewards. The study assigned different energy costs to each robotic arm, allowing the robot to learn how to adjust its movement based on cost, using less energy when the reward is small and exerting more effort if the reward is appropriate [39]. MARL with Deep Q-learning was also used in traffic signal control. Applied a cooperative multi-agent DQN framework to manage six interconnected intersections, where each intersection acted as an independent learning agent. Using the SUMO simulator, each agent selected signal phases to minimize congestion and improve overall traffic flow. Their approach introduced a reward function based on the standard deviation of stopping vehicles across lanes, showing balanced lane utilization rather than simply reducing queue lengths. This design showed that MARL with DQN can enhance not only traffic efficiency but also environmental sustainability by reducing emissions and fuel consumption. In businesses in developing countries, it is essential that growth should be the top priority; however, this creates a problem in increasing taxation for improving infrastructure to further support the current population. Using MARL, we can create a model system where multi-agent RL acts as different parties that discover the optimal tax policies without needing to rely on traditional economic models [66].

2.6 Application to Other Game Environments

Although this study focuses on Stratego as the game environment, the MARQ framework can also be applied to other strategic and simulation-based games that involve decision-making and hidden information. Sports management titles such as Madden NFL's Dynasty Mode, NBA 2K's franchise mode, Esports Manager, and Football Manager feature complex systems of player development, transfer, and long-term planning, where agents must act without complete information about the future player performance and market behavior. In these environments, the MARQ framework could model AI agents capable of learning optimal trade strategies or budget allocations through reinforcement learning. The framework's belief-state modeling would allow the AI to check hidden player attributes or opponent strategies, while the Deep Q-Network could optimize sequential decisions across multiple simulated seasons.

Similarly, the framework could be applied to a game called Clash Royale, where players must make fast tactical decisions without full knowledge of their opponent's hand or resource level. The Probabilistic Belief State in this setting could estimate the likelihood of specific opponent cards being available, while the Deep Q-Network would select optimal actions for troop deployment and timing.

IS-MCTS could be used to search over possible opponent hands, enabling informed decision-making for lane control or counter-attack scenarios.

Chapter 3

Theoretical Frameworks

3.1 Stratego Game Domain

Stratego is an imperfect information board game with European roots, popularized in the Netherlands in the 1940s and released in North America in 1961 [4]. The game is played on a 10x10 board that includes two 2x2 "lake" areas in the center, which are impassable. Each player commands an army of 40 pieces of varying ranks, and during the game, the identity and rank of the pieces are kept secret from the opposing player. The objective is to capture the opponent's flag or eliminate all of their movable pieces.

Each army consists of 12 different types of pieces, with ranks ranging from 1 (the Spy) to 10 (the Marshal), alongside Bombs and a Flag. Higher-ranked pieces defeat lower-ranked ones during combat, except in special situations: the Spy can defeat a Marshal if attacking first, the Miner is the only piece that can defuse Bombs, and Scouts can move multiple squares in a straight line. Victory is achieved by capturing the opponent's Flag, immobilizing all movable enemy pieces, or forcing the opponent to run out of time.

The high theoretical state space of Stratego shows how difficult the game is to solve:

$$|\mathcal{S}| = \binom{40}{1,1,2,3,4,4,4,5,8} \times 10^{10} \times 2^{40} \approx 10^{535} \tag{3.1}$$

Perolat et al. [45] established this complexity estimate. Because there are so many states, the agent must use reasoning with limited knowledge and manage hidden information rather than trying

Figure 3.1: Stratego Pieces

to look at every possible outcome.

3.2 MARQ Framework Overview

The MARQ framework, a Multi-Agent Rainbow Deep Q-Networks system, uses self-play and deep reinforcement learning to train agents for Stratego's large state space and hidden information [11]. The framework uses a league-based self-play system where agents learn by competing against many different opponents, including their own previous versions and exploratory variants.

This approach helps solve the main challenge of Stratego: thinking about hidden information without trying to calculate every single possibility. While the number of possible game states exceeds 10^{535} and initial setups reach 10^{66} per player [45], the MARQ framework handles this through implicit belief representations, probabilistic reasoning, and pattern recognition. The framework combines deep reinforcement learning to estimate values, attention mechanisms to process game history, and mixed strategies to prevent being beaten, all while keeping the calculations fast enough to run.

The MARQ framework works by connecting its different parts together. It starts with observa-

Figure 3.2: Stratego Game board.

tion processing, where the AAREN module takes raw game positions and moves to create learned embeddings that help guess piece identities. These embeddings are the base for all subsequent reasoning. The whole architecture uses a league-based training method where MARQ agents are trained and tested at the same time.

3.3 MARQ Policy Definition

Policy optimization in reinforcement learning means slowly improving an agent's strategy to get the highest total reward [54]. In games with hidden information and multiple players, we need policies that work well against set opponents and can also handle uncertainty and changing strategies from other agents. The MARQ framework solves this by connecting belief state reasoning with value-based decisions.

The policy learning process in MARQ has a few goals. The main goal is to win games through tactical choices. The second goal is to improve the quality of the embeddings created by the AAREN module. These goals are optimized together, as better embeddings help with decision-making, and good decisions reveal opponent information that helps improve the embeddings.

Table 3.1: Key Symbols and Descriptions from the MARQ Framework

Symbol	**Description**	**Example (MARQ Framework)**
$\pi_i(a\|I_i)$	Policy for agent i choosing action a given information set I_i	Probability of moving a piece forward given current visible board state and belief about opponent pieces
$\mathcal{B}_t$	Implicit belief representation at time t	Probability distribution over possible identities and positions of hidden opponent pieces inferred by AAREN
$v_i^\pi(h, \mathcal{B}_t)$	Value function for agent i following policy π given history h and belief $\mathcal{B}_t$	Expected reward for current board position considering uncertainty about opponent setup
$Q_{\text{network}}(s', a)$	Deep Q-Network approximation of action-value function	Neural network estimate of value for attacking with a suspected Marshal vs opponent piece
$R^{\text{capture}}i, t+1$	Immediate reward from piece captures weighted by piece values	+9 points for capturing opponent Marshal, -1 for losing a Scout
$O = \{o_1, \ldots, o_t\}$	Observation sequence processed by AAREN attention mechanism	Sequence of board states, moves, and attack outcomes throughout the game
α_i	Attention weights for historical observations in AAREN	Higher weight on turn when opponent's Marshal was revealed, lower on routine Scout moves
π^*	Nash equilibrium policy profile	Optimal mixed strategy that cannot be exploited by opponent strategy changes
$Q_{eval}(\mathcal{B})$	Embedding quality evaluator output	Confidence score for learned representation accuracy
$H(\pi)$	Policy entropy measure	Ensures minimum unpredictability to prevent exploitation
ϵ	Exploration rate for agent behavior	Controls exploration vs exploitation balance in multi-agent learning
γ	Discount factor for future rewards	Importance of long-term vs immediate rewards in value estimation

3.3.1 Core Policy Framework

A policy in the MARQ framework, denoted as $\pi_i(a|I_i)$, defines the probability distribution over actions for agent i conditioned on its current information set I_i. Unlike policies in perfect information games that operate on complete game states, MARQ policies must reason under uncertainty about the true state of the environment. The information set I_i contains all observations available to the agent, including visible board positions, revealed piece identities, and the history of observed movements and combat outcomes.

The main challenge in games with hidden information is that agents must make decisions without knowing the opponent's private information. In Stratego, this includes the rank and position of any opponent pieces that have not been in a battle yet. Therefore, the policy must include guesses about the opponent's pieces in its decision process and choose actions that work well across many possible hidden states.

To solve this, the MARQ policy uses implicit belief representations $\mathcal{B}_t$ rather than trying to list every possible game state. These representations are learned embeddings that the AAREN module updates as more information is seen. The policy is defined as:

$$\pi_i(a|I_i, \mathcal{B}_t) = P(\text{action} = a \mid \text{information set } I_i, \text{implicit belief } \mathcal{B}_t) \tag{3.2}$$

This shows that playing well in these games requires reasoning about hidden states. The policy learns to weight actions based on their expected value across the inferred distribution, rather than just optimizing for one specific opponent setup.

3.3.2 Implicit Representation Optimization

The quality of the implicit belief representation directly impacts decision quality, as inaccurate inferences lead to poorly calibrated action values. MARQ therefore incorporates representation accuracy as an auxiliary training objective, using revealed piece identities as supervised labels for embedding refinement.

When combat reveals the piece identities, the system calculates the prediction error using Kullback-Leibler (KL) divergence.

$$\mathcal{E}_{\text{pred}}(t) = \sum_{p \in \text{Pieces}_{\text{revealed}}(t)} D_{\text{KL}}\left(\text{Representation}_t(p) \| \delta_{s_p^*}\right) \tag{3.3}$$

Here, $\delta_{s_p^*}$ denotes the Dirac delta distribution. This error quantifies the difference between the predicted and actual piece identities.

This prediction error is transformed into a representation accuracy reward signal that encourages the policy to develop embeddings that better predict actual piece configurations:

$$R_{\text{accuracy}}(t) = -\mathcal{E}_{\text{pred}}(t) = -\sum_{p \in \text{Pieces}_{\text{revealed}}(t)} \left[\text{Representation}_t(p = s_p^*) > 0\right] \cdot \log \text{Representation}_t(p = s_p^*) \tag{3.4}$$

The optimal policy is then defined as the policy that maximizes the joint objective of game outcomes and representation accuracy:

$$\pi_i^* = \arg\max_{\pi_i} \mathbb{E}_{\tau \sim \pi_i} \left[\sum_{t=0}^{T} \gamma^t \left(R_{\text{game}}(t) + \eta \cdot R_{\text{accuracy}}(t) \right) \right] \tag{3.5}$$

The hyperparameter $\eta \in \mathbb{R}^+$ controls the relative importance of representation accuracy versus immediate game rewards.

The value function in MARQ extends the standard Bellman equation to include how beliefs change over time. This equation expresses the expected cumulative reward as a function of the current information set and implicit belief.

$$v_i^\pi(I, \mathcal{B}_t) = \mathbb{E}_\pi \left[R_{i,t+1}^{\text{capture}} + \lambda_1 R_{i,t+1}^{\text{info}} + \lambda_2 R_{i,t+1}^{\text{position}} + \gamma v_i^\pi(I_{t+1}, \mathcal{B}_{t+1}) \mid I_t = I, \mathcal{B}_t \right] \tag{3.6}$$

The reward function is decomposed into three strategic components. The capture reward R^{capture} reflects material changes, R^{info} quantifies the value of information revelation, and R^{position} measures territorial control.

The goal of MARQ training is to reach a Nash equilibrium, where no player can improve their expected payoff through unilateral deviation. This strategic stability ensures that the policy remains robust against adversarial exploitation.

$$v_i(\pi^*) = \max_{\pi_i} v_i(\pi_i, \pi_{-i}^*) \tag{3.7}$$

Finding exact Nash equilibria in large imperfect information games is computationally intractable, so MARQ instead seeks approximate equilibria through self-play training with diverse opponent populations.

3.4 Deep Q-Network Architecture

Deep Q-Networks in the MARQ framework operate on implicit belief representations rather than complete game states [54]. The network approximates Q-values for probability distributions over opponent configurations, $\mathcal{B}_t$:

$$Q(s, a) = \mathbb{E}_{s' \sim \mathcal{S}_t \subseteq \mathcal{B}_t}[Q_{network}(s', a)]$$

The system uses a standard replay buffer for experience storage, sampling transitions $(\mathcal{B}_t, a_t, r_t, \mathcal{B}_{t+1})$ uniformly to train the value function.

3.4.1 Rainbow DQN Components

The implementation incorporates key enhancements from the Rainbow DQN architecture [26] to improve stability and sample efficiency.

Distributional RL (Categorical DQN). Categorical DQN (C51) models the value distribution directly to capture the inherent uncertainty of the game. The distribution is approximated by 51 atoms that span the range from V_{min} to V_{max}.

$$Z_\theta(s, a) = \sum_{i=1}^{N} p_i(s, a)\delta_{z_i} \tag{3.8}$$

The training objective minimizes the Kullback-Leibler divergence between the predicted distribution and the projected target distribution:

$$D_{KL}(\Phi\hat{\mathcal{T}} Z_{\theta'} || Z_\theta) \tag{3.9}$$

where Φ is the projection operator mapping the Bellman update onto the fixed support atoms.

Noisy Networks for Exploration. The framework utilizes Noisy Linear layers to facilitate consistent exploration. By adding learned noise to the network weights, the agent maintains an unpredictable strategy without requiring a separate exploration schedule. The network optimizes the degree of exploration required for different states.

$$y = (\mu_w + \sigma_w \odot \epsilon_w)x + (\mu_b + \sigma_b \odot \epsilon_b) \tag{3.10}$$

Dueling Network Architecture. The network explicitly separates the estimation of state values $V(s)$ and advantage values $A(s,a)$ using two separate streams that share a common convolutional feature extractor:

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + \left(A(s,a;\theta,\alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s,a';\theta,\alpha) \right) \tag{3.11}$$

This decomposition leads to better policy evaluation in the presence of many similar-valued actions.

The Q-values are augmented with an exploration bonus based on the uncertainty of the inferred representation, utilizing an optimism in the face of uncertainty approach.

$$Q_{augmented}(s,a) = Q_{base}(s,a) + \lambda \cdot U(a) \tag{3.12}$$

where $U(a)$ represents the entropy of the learned distribution at the target location, encouraging the agent to investigate unknown opponent pieces.

3.4.2 Training Stability via Regularization

Recent work, like the Ataraxos system which reached a superhuman level in no-press Diplomacy, shows that keeping training stable is just as important as the algorithm itself [52]. Deep reinforcement learning agents often face two main problems: spending too much time on experiences with little information, and jumping between different strategies without finding a robust one. The MARQ framework uses two shared methods to keep training stable.

Advantage Filtering for Sample Efficiency. Advantage filtering helps the agent learn faster from fewer games [57]. In reinforcement learning, agents learn by comparing their predicted values with outcomes. The temporal-difference (TD) error measures this difference. Usually, a replay buffer samples past games randomly, treating all moves the same even if some are more useful for learning. Advantage filtering focus training on moves with high errors by sampling the buffer and keeping only the most informative ones:

$$\mathcal{B}_{filtered} = \mathrm{Top}_{n/k}\left(\mathcal{B}_{sampled}, |\delta_i|\right) \tag{3.13}$$

where $\delta_i = r_i + \gamma^n V(s'_i) - V(s_i)$ is the n-step TD error for transition i.

Dynamic Damping with Power-Law Annealing. To address strategy cycling, the loss function incorporates two regularization terms based on Kullback-Leibler divergence:

$$\mathcal{L}_{total} = \mathcal{L}_{C51} + \alpha(t) \cdot D_{KL}(\pi \| \pi_{uniform}) + \beta(t) \cdot D_{KL}(\pi_{target} \| \pi) \tag{3.14}$$

The coefficients follow power-law schedules with opposing dynamics:

$$\alpha(t) = \alpha_0 \cdot (1 - t/T)^p \tag{3.15}$$

$$\beta(t) = \beta_0 + (\beta_T - \beta_0) \cdot (t/T)^p \tag{3.16}$$

where t is the current episode, T is the total training duration, and $p = 2$ produces quadratic curves. This schedule ensures the agent explores a lot early on when it is uncertain, and moves to a more stable strategy as the policy matures.

3.5 AAREN and Learned Embedding System

The AAREN module (Attention as Recurrent Neural Network) [23] is the main memory and history system within MARQ. Instead of keeping simple counts of piece identities, AAREN creates learned embeddings from opponent moves that the agent interprets through training. This method is made to handle long sequences of information in Stratego, remembering details over hundreds of moves while avoiding the gradient problems that affect older models like LSTMs. AAREN uses an attention mechanism to look back at the whole list of moves, making it efficient at gathering history [23].

3.5.1 Recurrent Attention Computation

AAREN implements an attention-based recurrent computation that maintains a state triplet (a_t, c_t, m_t) across the observation sequence. Unlike traditional recurrent neural networks that suffer from vanishing gradients over long sequences, AAREN provides direct access to the entire history through attention weights while maintaining O(1) inference complexity per timestep.

Given a sequence of observations $O = \{o_1, \ldots, o_t\}$ where each o_i encapsulates the visible board state and public action outcomes, the AAREN cell first projects each observation into key and value representations:

$$k_t = W_k(o_t), \quad v_t = W_v(o_t) \tag{3.17}$$

The projection matrices W_k and W_v are learned during training. The key k_t determines how relevant the current observation is for belief updates, while the value v_t encodes the informational content to be integrated into the belief representation. The attention score s_t measures the relevance of the current observation relative to the learned query vector q:

$$s_t = q^T k_t \tag{3.18}$$

The weighted value accumulator a_t aggregates value representations across the sequence:

$$a_t = a_{t-1} \exp(m_{t-1} - m_t) + v_t \exp(s_t - m_t) \tag{3.19}$$

The term $\exp(m_{t-1} - m_t)$ rescales the previous accumulator when a new maximum is encountered, while $\exp(s_t - m_t)$ computes the relative attention weight for the current observation. The normalization constant $c_t = c_{t-1} \exp(m_{t-1} - m_t) + \exp(s_t - m_t)$ ensures that the effective attention weights sum to unity. The output representation $h_t = a_t/c_t$ represents an attention-weighted aggregation of all observations up to time t.

3.5.2 Piece Type Inference

The AAREN model learns to infer piece values from action sequences using 24-dimensional feature vectors encoding movement patterns, contextual information, and game state features. The network outputs probability distributions over piece types:

$$P(\text{piece\_type} = t \mid \text{action\_sequence}) = \text{softmax}(W_{out} \cdot h_T + b_{out})_t \tag{3.20}$$

This allows the network to use AAREN embeddings along with game rules and the board state to make better decisions under uncertainty.

The MARQ framework has an evaluation system that checks how good the embeddings are and helps them improve. The evaluator uses a neural network that takes embeddings as input and outputs quality scores $Q_{eval}(\mathcal{E}) = \text{MLP}(\mathcal{E}; \theta_{eval})$. The training is based on comparing the guesses to the true piece identities:

$$R_{quality}(\mathcal{B}, g) = \alpha \cdot \mathcal{B}[g] - \beta \cdot |V_{pred} - V_{actual}| - \gamma \cdot \text{distance\_penalty} \tag{3.21}$$

The system determines when additional information gathering is beneficial through uncertainty assessment:

$$\text{NeedMoreInfo}(\mathcal{B}) = \nVdash\{H(\mathcal{B}) > \theta_H\} \cdot \nVdash\{Q_{eval}(\mathcal{B}) < \theta_Q\} \tag{3.22}$$

3.5.3 Embedding Updates on Piece Revelation

When a battle reveals a piece's true type, the AAREN embeddings synchronize with ground truth. Upon revealing piece type g at position pos, the inferred distribution collapses to certainty:

$$\text{Representation}_{pos}[t] = \begin{cases} 1.0 & \text{if } t = g \\ 0.0 & \text{otherwise} \end{cases} \tag{3.23}$$

The revealed piece count is incremented, and remaining beliefs are adjusted to respect piece count constraints. For unrevealed positions $p \neq pos$:

$$\mathcal{B}_p[t] \leftarrow \mathcal{B}_p[t] \cdot \frac{\text{max\_count}[t] - \text{revealed\_count}[t]}{\text{remaining\_count}[t]} \tag{3.24}$$

For example, when the single Marshal is revealed, the probability of any other piece being a Marshal drops to zero.

3.5.4 Hindsight-Aided Belief Refinement

While the AAREN module provides inference during gameplay, the Hindsight-Aided Belief Refinement (HABR) module enables learning from verified information. When a piece is revealed through combat, HABR applies retrospective learning across the entire history of that piece, using the ground truth as a supervisory signal for past predictions.

Information Gain Reward. The Information Gain reward quantifies the value of observations that reduce uncertainty about opponent pieces. Let $H(P)$ denote the Shannon entropy of a probability distribution P. The KL divergence from a uniform prior U over N piece types is calculated as follows.

$$D_{KL}(P \parallel U) = \log(N) - H(P) \tag{3.25}$$

The Information Gain reward measures the change in this divergence between consecutive representation updates:

$$R_{gain}(t) = D_{KL}(\text{Rep}_{t-1} \parallel U) - D_{KL}(\text{Rep}_t \parallel U) \tag{3.26}$$

Substituting the divergence formula yields:

$$R_{gain}(t) = (\log(N) - H(\text{Rep}_{t-1})) - (\log(N) - H(\text{Rep}_t)) = H(\text{Rep}_{t-1}) - H(\text{Rep}_t) \tag{3.27}$$

This result shows that R_{gain} equals the reduction in entropy. A positive reward is achieved only when the observation at time t increases certainty about piece identity compared to $t-1$. This aligns the agent's incentives with active information gathering.

Sinkhorn Constraint Normalization. Independent belief updates for each piece position can lead to inconsistent global predictions. For example, if piece count constraints are not enforced, the model might assign high probability to multiple pieces being the single Marshal. This phenomenon, termed identity inflation, violates the physical constraints of the game.

The Sinkhorn normalization addresses this through iterative proportional fitting. Given a representation matrix $M \in \mathbb{R}^{P \times S}$ where P is the number of tracked pieces and S is the number of piece types, two constraints must hold: the row constraint $\sum_s \text{Rep}(p,s) = 1$ ensures each piece has exactly one identity, and the column constraint $\sum_p \text{Rep}(p,s) = Count(s)$ ensures the total predicted count of each piece type matches the remaining count.

The Sinkhorn algorithm alternates between row and column normalization until it finishes. This approach is differentiable, allowing gradients to pass through. When piece A is confirmed as the Spy, the Sinkhorn layer automatically updates all other pieces, creating a shared update across the entire inferred state.

Retrospective KL Divergence. When a piece with identity s_p^* is revealed at time t, the retrospective loss applies across the entire history of that piece:

$$\mathcal{L}_{retro} = \sum_{k=1}^{t} \lambda^{t-k} D_{KL}(\text{Rep}_k(p) \parallel \delta_{s_p^*}) \tag{3.28}$$

where $\delta_{s_p^*}$ is the Dirac delta distribution concentrated on the true piece type. The temporal decay factor $\lambda < 1$ (typically 0.9) assigns higher weight to recent predictions.

The decay factor addresses the non-stationary nature of piece behavior in Stratego. A piece may exhibit deceptive behavior in the early game, such as a Marshal moving conservatively to avoid revealing its identity, before adopting more characteristic patterns in the endgame. By weighting recent observations more heavily, the loss function prioritizes learning from high-information behaviors near the revelation event while still providing a smoothed learning signal over the piece's history.

3.6 Multi-Agent Learning and Strategy Mixing

Multi-agent learning in zero-sum games requires stable training architectures to manage evolving strategy distributions. MARQ employs a league-based system for this purpose.

3.6.1 League-Based Training Architecture

The league has a changing group of agents to help the model learn to play against many different strategies [53]. The main agent is the best current policy being trained. The historical pool has old versions of the main agent that represent different stages of training.

During training, opponents are sampled using a specific probability distribution.

$$P(\text{opponent}) = \begin{cases} 0.5 & \text{Main Agent (Self-Play)} \\ 0.5 & \text{Uniform(Historical Pool)} \end{cases} \tag{3.29}$$

This keeps the agent from looping through the same strategies over and over, forcing it to defeat all previous versions of itself.

3.6.2 Strategy Mixing for Exploitation Resistance

Extensive-form imperfect information games with deterministic strategies are inherently suboptimal and vulnerable to exploitation [45, 55]. An adversary with knowledge of a deterministic policy effectively reduces the game to a perfect information setting, nullifying the strategic uncertainty fundamental to games like Stratego. The MARQ framework addresses this challenge through two complementary mechanisms.

At the action selection level, MARQ employs Noisy Linear layers that inject learned, state-dependent noise into the policy. Rather than using fixed exploration schedules such as ϵ-greedy, the network learns the appropriate degree of randomization for each game state.

At the strategy level, MARQ achieves mixing through diverse opponent exposure during training. The league system maintains a population of historical agent checkpoints representing different strategic approaches discovered during training. The combination of action randomness and strategy variety produces policies that are unpredictable and hard to exploit.

3.6.3 Multi-Dimensional Reward Structure

The reward function $R(s, a, s')$ is shaped to guide learning in the sparse-reward Stratego environment [20]:

$$R_{total} = R_{move} + R_{capture} + R_{tactical} + R_{info} \tag{3.30}$$

The move reward R_{move} provides small rewards for advancing pieces (approximately +0.05) to encourage active play. The capture reward $R_{capture}$ is scaled by the rank of the captured piece according to $0.15 \times \frac{\text{Rank}}{11.0}$. The tactical reward $R_{tactical}$ gives specific bonuses for key strategic captures, including +0.5 for Miner vs Bomb defusals and +1.0 for Spy vs Marshal assassinations. The information reward R_{info} provides +0.3 for revealing high-value opponent pieces.

To facilitate the transition from tactical learning to strategic gameplay, the framework employs *Reward Annealing*. Dense shaping rewards, such as material and information gains, are scaled down as the training curriculum advances. In later training phases, these auxiliary signals are completely annealed out, forcing the agent to rely exclusively on sparse, terminal win/loss rewards. This progression prevents the agent from developing sub-optimal local behaviors driven by short-term shaping incentives.

3.7 Specialized Agent Architectures

3.7.1 Setup Agent via Distributional RL

The game's initial setup is handled by a specialized SetupAgent that learns optimal piece configurations by treating the placement phase as a sequential decision process. The state representation consists of a 3-channel input tensor encoding the current board state with already-placed pieces, a valid positions mask indicating legal placement locations, and a current piece type mask identifying which piece is being placed.

The agent outputs a distributional Q-value for each of the 100 board positions, mapping $Q(s, pos)$ to a distribution over $[V_{min}, V_{max}]$. This distributional approach captures the inherent uncertainty in setup quality, where the true value of a placement depends on the unknown opponent configuration.

To prevent creating static, easily countered setups, the agent includes an exploitability critic network that penalizes predictable placements:

$$\mathcal{L}_{total} = \mathcal{L}_{C51} + \lambda \cdot \mathcal{L}_{predictability} \tag{3.31}$$

3.7.2 Information Set Monte Carlo Tree Search (IS-MCTS)

For important decisions, especially in the endgame, MARQ uses an Information Set MCTS agent [16] that combines search with learned value functions. The search process has three phases.

First, during the determinization phase, the agent samples a consistent concrete world state s_{det} from the history-based beliefs according to $s_{det} \sim P(S|\mathcal{B}_t)$. This converts the imperfect information game into a perfect information instance for the duration of the simulation.

Second, during tree traversal, the search tree is navigated using a UCB1 variant to balance exploration and exploitation of move sequences. Nodes in the tree represent information sets rather than individual states, allowing the tree to be shared across multiple determinizations.

Third, during leaf evaluation, terminal or unexpanded nodes are evaluated using the Rainbow DQN value function rather than random rollouts:

$$V(s_{leaf}) \approx Q_{DRQN}(s_{leaf}, a_{best}) \tag{3.32}$$

3.8 Game Environment and State Management

The Stratego environment provides the foundation for agent training and evaluation, managing the complex dynamics of imperfect information.

3.8.1 Information Set Management

The environment maintains distinct representations for different information contexts:

$$\mathcal{I}_{player} = \{s : s \text{ is consistent with player observations}\} \tag{3.33}$$

Each player receives only the subset of game state information available through their observation function.

3.8.2 Temporal State Evolution

Game states evolve according to the update function.

$$s_{t+1} = \text{EnvStep}(s_t, a_t^1, a_t^2, \theta_t) \tag{3.34}$$

where θ_t represents stochastic elements (hidden information revelation, time limits, etc.).

3.8.3 Valid Action Filtering

The environment enforces game rules through valid action constraints:

$$\mathcal{A}_{valid}(s, player) = \{a : \text{IsLegalMove}(a, s, player) \wedge \text{RespectsGameRules}(a, s)\} \tag{3.35}$$

This ensures that agents can only select legal moves consistent with Stratego's rules and piece capabilities.

3.8.4 Piece Setup and Initialization

Strategic piece placement is governed by setup policies.

$$\pi_{setup}(\text{placement}|player) = \text{NeuralNetworkSetup}(\text{strategic\_features}, \theta_{setup}) \tag{3.36}$$

where strategic features include positional value maps, piece interaction potentials, and defensive vulnerabilities. How the pieces are placed at the start is a very important decision that affects the whole game [18].

3.9 MARQ Move Selection and Decision-Making

The culmination of the MARQ framework's architectural components is the real-time move selection process. This procedural logic, detailed in Algorithm 1, integrates learned positional priors with tactical analytical search to optimize behavior in adversarial environments.

Algorithm 1 MARQ: Decision-making and Move Selection

Input: Public Observations O, Board State S, DQN weights θ, AAREN weights ϕ, Temperature T
Output: Optimal Action a^*

$\mathcal{H}_t \leftarrow \text{UpdateAggregation}(O, \phi)$
$\mathcal{B}_t \leftarrow \text{GetSoftmaxBeliefs}(\mathcal{H}_t)$
$X_t \leftarrow \text{Concatenate}(S, \mathcal{H}_t)$
$Q(s, \cdot) \leftarrow \text{RainbowInference}(X_t, \theta)$
$\mathcal{A} \leftarrow \text{LegalMoves}(S)$
for all $a_i \in \mathcal{A}$ **do**
  if IsAttack(a_i, S) **then**
    $target\_pos \leftarrow \text{TargetSquare}(a_i)$
    $P \leftarrow \mathcal{B}_t(target\_pos)$
    $V_{exp} \leftarrow \sum_{r \in \mathcal{R}} P(r) \cdot \mathbb{U}(\text{Attacker}, r)$
    $V_{final}(a_i) \leftarrow \lambda Q(s, a_i) + (1 - \lambda) V_{exp}$
  else
    $V_{final}(a_i) \leftarrow Q(s, a_i)$
  end if
end for
if Training/Self-Play and $T > 0$ **then**
  $\pi(a_i) \leftarrow \frac{\exp(V_{final}(a_i)/T)}{\sum_{a \in \mathcal{A}} \exp(V_{final}(a)/T)}$
  return $a^* \sim \pi(\cdot)$
else
  return $a^* \leftarrow \arg\max_{a_i \in \mathcal{A}} V_{final}(a_i)$
end if

3.9.1 Historical Context Update

The first stage of the decision process involves the AAREN module's recurrent attention system. As game observations (moves and combat outcomes) are received, the history aggregator updates its internal state. This ensures that the agent's current embedding reflects the cumulative behavioral patterns of the opponent, rather than just the immediate board state.

3.9.2 Probabilistic Belief Extraction

Upon updating the history, the system extracts the softmax probability distributions for hidden pieces. These piece-identity predictions are generated for every square on the board, representing the agent's current belief state regarding the hidden ranks of unrevealed opponent pieces.

3.9.3 Information Set Tensorization

The visible board state and the learned AAREN embeddings are concatenated to form a high-dimensional information set representation. In the current implementation, this produces a 79-channel input tensor (15 physical board features + 64 history embedding channels), which encapsulates both the physical layout and the inferred hidden context.

3.9.4 Value Distribution Estimation

The 79-channel tensor is processed through the Rainbow DQN's convolutional backbone and C51 distributional head. This step estimates the Q-value distribution over 51 atoms for all possible actions, capturing the inherent multi-modal uncertainty of the state.

3.9.5 Action-Space Filtering

To ensure rule compliance and efficiency, the agent applies legal move masking. This step filters the output of the Q-network against the set of valid moves $\mathcal{A}$ defined by Stratego's rules, preventing the selection of illegal actions.

3.9.6 Hierarchical Policy Blending

For defensive and positional moves, the agent relies on the DQN's positional priors. However, for offensive actions, the agent employs a hierarchical blend: the score from the neural network is weighted against the analytical outcome of the search. This prevents "hallucinated" aggression by grounding neural predictions in tactical reality.

3.9.7 Expectamax Search and Tactical Summation

While standard adversarial search algorithms like Minimax assume an opponent who plays optimally to minimize the agent's utility, many environments involve stochastic elements or hidden information that are better modeled as probabilistic transitions. The *Expectamax* algorithm, originally presented by Michie [40] and formalized by Ballard [6], addresses this by introducing "Chance Nodes" into the game tree.

In the case of Stratego, the identity of an opponent piece encountered during an attack is modeled as a stochastic variable. The Expectamax value is computed as an exact summation over the rank probabilities provided by the belief extractor:

$$\mathbb{E}[V] = \sum_{r \in \mathcal{R}} P(\text{target is rank } r) \cdot \mathbb{U}(\text{outcome vs } r) \tag{3.37}$$

This allows for tactical aggression when the probability of a successful capture outweighs the risk of piece loss, grounded in the AAREN's inference history.

3.9.8 Stochastic Action Selection (Temperature Scaling)

To prevent the agent from collapsing into rigid, exploitable behavior loops during self-play and evaluation, the final action selection step incorporates Temperature Scaling (Boltzmann Options). Rather than systematically choosing the action with the absolute highest expected value via a hard *argmax* function, the agent adopts a stochastic softmax mechanism parameterized by a temperature variable T. For a given state, the probability of selecting action a_i is determined by the Boltzmann distribution over the action values V_{final}:

$$\pi(a_i) = \frac{\exp\left(V_{final}(a_i)/T\right)}{\sum_{a \in \mathcal{A}} \exp\left(V_{final}(a)/T\right)} \tag{3.38}$$

As T approaches zero, the selection strategy becomes deterministically greedy (equivalent to *argmax*). When T increases, near-optimal actions receive proportionally higher selection probabilities, injecting controlled variance. This stochasticity provides two key operational benefits: it generates a more diverse and unpredictable curriculum roster during league training, and it prevents tactical stagnation within recurring, indecisive positional maneuvers.

3.10 Generalizability

The MARQ framework is designed to provide both mathematical stability in adversarial settings and broad applicability to diverse real-world domains. This section details the convergence guarantees of the training methodology and the potential for applying the learned representations to non-gaming environments.

3.10.1 Mathematical Convergence Properties

The MARQ framework provides theoretical guarantees for convergence and performance under specific conditions.

Incremental Learning Bounds

The MARQ algorithm provides a specific bound on value function estimation:

$$\|V_t - V^*\|_\infty \leq \frac{C}{\sqrt{t}} + \epsilon_{approx} \tag{3.39}$$

where C depends on reward bounds and discount factor, and ϵ_{approx} represents function approximation error.

Implicit Representation Accuracy Bounds

The representation accuracy improves with additional observations according to the following relationship:

$$\mathbb{E}[\text{Accuracy}(\mathcal{B}_t)] \geq 1 - \exp(-\alpha \cdot N_{obs}(t)) \tag{3.40}$$

where $N_{obs}(t)$ is the cumulative number of informative observations and α is a positive constant.

Strategy Convergence in Self-Play

The league training methodology ensures convergence to ϵ-Nash equilibria under standard monotonic improvement assumptions:

$$\lim_{T\to\infty} \|\pi_T - \pi^*\| \leq \epsilon \tag{3.41}$$

for some $\epsilon > 0$ depending on the exploration parameters and opponent diversity. The league training system ensures improvement through hierarchical best-response maintenance, main-pool promotion thresholds, and diversity-driven exploration.

3.10.2 Applied Generalizability and Mission Impact

While we developed the MARQ framework for the game of Stratego, its architectural design is applicable to broader real-world problems involving partial observability and adversarial decision-making.

Inference in Degraded Environments

The combination of an occlusion-based curriculum and AAREN history aggregation provides a robust method for training agents to operate in degraded environments. In autonomous drone applications, for example, sensors may fail or be obstructed by smoke or weather. MARQ's ability to maintain an implicit belief state allows for continued navigation and objective completion.

Signal Intelligence and Pattern Discovery

In adversarial signal environments, such as electronic warfare, the implicit representation learning used in MARQ can be adapted to discover latent "identities" in high-dimensional signal data. By treating signal history as an action sequence, the network can learn to categorize threats without explicit labeling of every transmission.

Industrial Automation and Predictive Maintenance

MARQ's architecture can be used to integrate sparse sensor readings over time to predict hidden anomalies or maintenance needs in complex manufacturing pipelines, allowing for proactive adjustments before a system failure occurs.

These theoretical foundations and the adaptability of the learned embeddings provide confidence in the MARQ framework's ability to handle high-stakes strategic gameplay and its future transition to practical, real-world utility.

3.11 Algorithmic Evolution and Architecture Ablations

The final MARQ architecture resulted from systematic experimentation and the refinement of several key reinforcement learning components. Throughout the development process, multiple established algorithms were evaluated and subsequently replaced because of performance bottlenecks or insufficient strategic depth in the Stratego domain.

Value Optimization and Exploration. Early iterations of the agent utilized a vanilla DQN with ϵ-greedy exploration. This approach proved inadequate for the sparse-reward nature of Stratego and led to early stagnation. The implementation was upgraded to a Rainbow DQN architecture that incorporates Noisy Networks for state-dependent exploration. Furthermore, the standard double-DQN configuration was replaced with a Dueling Head architecture, which significantly improved value estimation accuracy in states where multiple actions provide similar utility. Additionally, to mitigate the risk of overfitting to deterministic opponent policies during self-play, the system employs *Boltzmann Exploration* (Temperature Scaling). Rather than acting purely greedily, the opponent's action selection process utilizes a temperature-scaled softmax over Q-values, injecting controlled stochasticity to generate a more robust and unpredictable training curriculum.

History and Memory Representation. Initial attempts to manage hidden information relied on explicit Probabilistic Belief States. However, these states introduced prohibitive computational complexity and memory overhead because they required $O(N^2)$ updates that throttled training throughput. The framework subsequently moved to an implicit belief system powered by the AAREN architecture. Similarly, the use of Long Short-Term Memory cells for trajectory embeddings was ablated in favor of the attention-based recurrence of AAREN, which provided more stable gradient flow over long sequences and better handled the non-Markovian nature of piece identity inference.

Architectural Refinements. The convolutional backbone underwent several iterations, moving from standard deep convolutional stacks to a dedicated ResNet architecture. Extensive testing identified 6 Residual Blocks as the optimal depth. This configuration provides full receptive field coverage of the 10x10 board while maintaining low inference latency. Optimization was transitioned from the standard ADAM optimizer to ADAMW to improve weight decay and policy loss stability.

Strategic Initialization and Efficiency. The piece placement phase was initially modeled using an agentic SetupAgent trained through distributional reinforcement learning. While functional, this approach introduced significant exploitability risks and operational overhead. The project transitioned to a non-agentic, fixed Heuristic Setup Agent with randomized flag positioning. This change resolved the exploitability concerns and provided substantial computational benefits. While the agentic setup required approximately 2 seconds per episode, the heuristic method completes in under 100 milliseconds. This optimization effectively halved the total training compute requirement by eliminating the massive overhead of the setup training phase.

Chapter 4

Methodology

This chapter explains how we tested the MARQ framework's ability to solve games with hidden information. We built a competitive system and then tested its win-rate against human players and its adaptability across different games. The following sections describe the steps we took.

4.1 Project Planning

We conducted this research using a plan that includes choosing participants and setting up the experiment. We used a rapid prototyping method to design and build the MARQ framework. The development phase ran from September 2025 to February 2026, including building the game environment and training the agent. User testing took place in March 2026, and we analyzed the data in April 2026.

4.1.1 Community Introduction to Stratego

We introduced Stratego to the participants first because none of them knew the rules or strategies. Stratego is a niche game, so there weren't many experienced players available to test against our model. By teaching everyone the game from scratch, we ensured that both the humans and the AI were learning at the same time, making the test fairer.

We selected 20 participants based on their availability and experience with strategy games. We looked for people from different backgrounds who enjoyed strategic thinking but didn't know how to play Stratego. This ensured they had the skills needed to play well while starting fresh with this specific game, allowing us to maintain a consistent starting point for the study.

4.1.2 Experimental Procedures

One-on-one matches between human participants and the MARQ framework will be conducted on-site using a single device. To ensure consistency, each participant will receive a checklist outlining the procedures for interacting with the game.

Each session will last one (1) hour, during which participants must complete at least two (2) games against the MARQ framework. Additional games may be played if the time permits. The following game statistics will be recorded will include the winner, total game duration, total moves per player, and the number and type of remaining pieces for both players. Immediately following the session, each participant will be given an evaluation form to provide open-ended feedback on their experience.

4.2 System Development

The system development consists of two components: the game environment and the MARQ framework. The game environment was developed to serve as the interface for human players and the world in which the trained agent operates. The MARQ framework provides the trained agent that competes against human players.

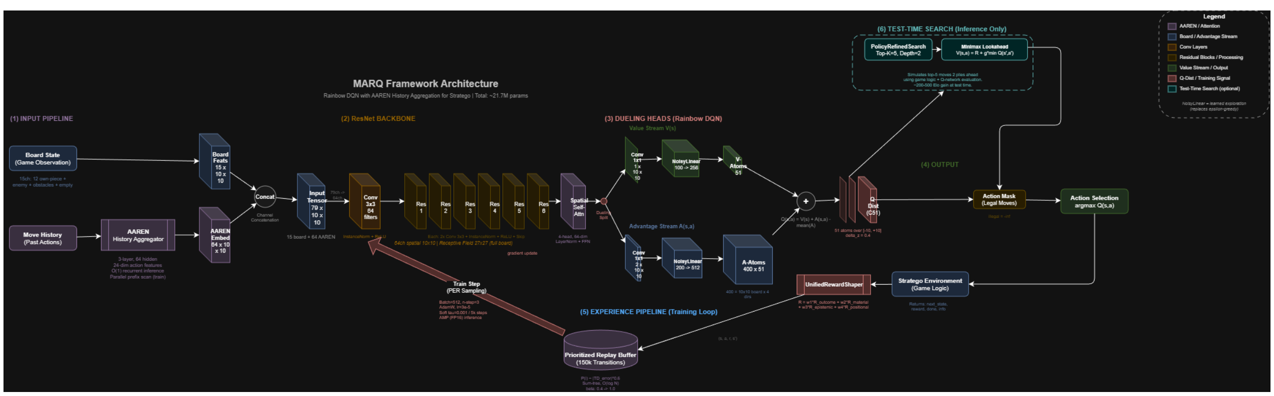


Figure 4.1: MARQ Framework Architecture. The input pipeline processes board states and move history through AAREN to produce implicit belief embeddings. These are concatenated with board features and fed into a ResNet-based Rainbow DQN with Spatial Attention and Dueling Heads.

4.2.1 Piece Value Computation

The reward function requires a quantitative assessment of piece value to guide learning. This section presents an analytical method for computing piece values based on probability theory, providing a deterministic alternative to empirical estimation through gameplay simulation.

Combat Dominance Score

The Combat Dominance Score (CDS) represents the probability that a given piece type defeats a uniformly random opponent piece. Let $\mathcal{P}$ denote the set of all piece types and n_i the count of piece type i in standard Stratego. The CDS for piece p is defined below.

$$\text{CDS}(p) = \frac{\sum_{i \in \mathcal{P}} \mathbb{1}_{p \succ i} \cdot n_i}{N} \tag{4.1}$$

where $N = \sum_{i \in \mathcal{P}} n_i = 40$ is the total piece count, $\mathbb{1}_{p \succ i}$ is an indicator function equal to 1 if piece p defeats piece i according to Stratego combat rules, and 0 otherwise. The combat rules specify that higher-ranked pieces defeat lower-ranked pieces, with specific exceptions. The Spy defeats the Marshal when attacking, and the Miner defeats Bombs.

Survival Rate

The Survival Rate (SR) measures the expected probability of a piece surviving an attack from a uniformly random attacker. Let $\mathcal{M} \subset \mathcal{P}$ denote the set of movable piece types (excluding Flag and Bomb). The SR for piece p is calculated as follows.

$$\text{SR}(p) = \frac{\sum_{a \in \mathcal{M}} \mathbb{1}_{p \succeq a} \cdot n_a}{\sum_{a \in \mathcal{M}} n_a} \tag{4.2}$$

The indicator function equals 1 if piece p survives an attack from piece a.

Auxiliary Value Components

Three additional factors contribute to piece value:

Strategic Ability Bonus. Certain pieces possess capabilities that extend beyond their combat rank. Table 4.1 enumerates these bonuses.

Table 4.1: Strategic ability bonuses for pieces with special capabilities.

Piece	**Bonus**	**Description**
Miner	2.0	Sole piece capable of removing Bombs
Spy	1.5	Defeats Marshal when initiating attack
Scout	1.0	Multi-square movement capability
Marshal	0.5	Highest combat rank

Mobility Factor. The Scout receives a mobility bonus of 1.0 due to its ability to move multiple squares per turn; all other movable pieces receive a base mobility of 0.5. Immovable pieces (Flag, Bomb) receive 0.

Scarcity Factor. The scarcity of each piece type influences its strategic importance. The scarcity factor is computed using logarithmic scaling.

$$\mathrm{SF}(p) = \log\left(\frac{\max_{i \in \mathcal{P}} n_i}{n_p}\right) + 1 \tag{4.3}$$

Composite Value Function

The final piece value is a weighted linear combination of the previous components.

$$V(p) = \alpha_1 \cdot \mathrm{CDS}(p) + \alpha_2 \cdot \mathrm{SR}(p) + \alpha_3 \cdot S(p) + \alpha_4 \cdot M(p) + \alpha_5 \cdot \mathrm{SF}(p) \tag{4.4}$$

where $S(p)$ is the strategic ability bonus and $M(p)$ is the mobility factor. The weights $\alpha = (0.40, 0.25, 0.20, 0.10, 0.05)$ are selected to prioritize combat capability while accounting for strategic factors. Values are normalized such that $V(\text{Scout}) = 1.0$, analogous to the convention in chess where the Pawn serves as the unit of measurement.

These values inform the material reward component of the reward shaping function described in subsequent sections.

4.2.2 Using AAREN to Remember Game History

The primary challenge in Stratego involves decision-making under partial observability. Rather than utilizing explicit piece counts, the framework employs architectural patterns inspired by DeepNash [45]. The Attention-as-a-Recurrent-Neural-Network (AAREN) module aggregates history and generates embeddings derived from opponent maneuvers to facilitate piece identity inference.

Table 4.2: Computed piece values with associated combat and survival metrics.

Piece	Value	CDS	SR
Marshal	8.4	0.825	0.939
General	8.0	0.800	0.939
Colonel	7.5	0.750	0.879
Major	6.7	0.675	0.788
Captain	5.7	0.575	0.667
Lieutenant	4.8	0.475	0.545
Miner	4.3	0.400	0.273
Sergeant	3.8	0.375	0.424
Bomb†	2.0	—	—
Spy	1.1	0.050	0.000
Scout	1.0	0.050	0.030
Flag‡	0.1	—	—

†Immovable defensive piece; eliminates any non-Miner attacker.
‡Immovable objective piece; capture results in game loss.

To mitigate the "sparse death" problem, where a lack of observation history produces uninformative zero tensors, a learnable default embedding is integrated. This provides a consistent initialization for the network and maintains training stability during initial game states.

Action Feature Extraction. For each observed opponent action, AAREN extracts a 24-dimensional feature vector.

- **Movement features**: Direction (normalized), distance, whether the move was an attack
- **Position features**: Starting position (normalized), board region indicators
- **Behavioral indicators**: Scout move flag (distance > 1), forward advancement, lateral movement
- **Temporal features**: Turn number, piece activity (historical movement frequency)

Recurrent Embedding Generation. The AAREN module processes action sequences using a parallel prefix scan algorithm during training for computational efficiency, and $O(1)$ recurrent updates during inference for real-time play. For each enemy piece position with observed history, AAREN maintains a hidden state that captures temporal patterns in the piece's behavior. This hidden state is output as a 64-dimensional embedding vector per board position.

End-to-End Training. Unlike separate belief state modules that require explicit supervision, AAREN learns jointly with the Rainbow DQN through shared gradient flow. The combined optimizer updates both the AAREN encoder and the Q-network simultaneously, allowing the system to discover representations that are directly useful for action selection rather than optimizing for auxiliary prediction tasks.

4.3 Model Training

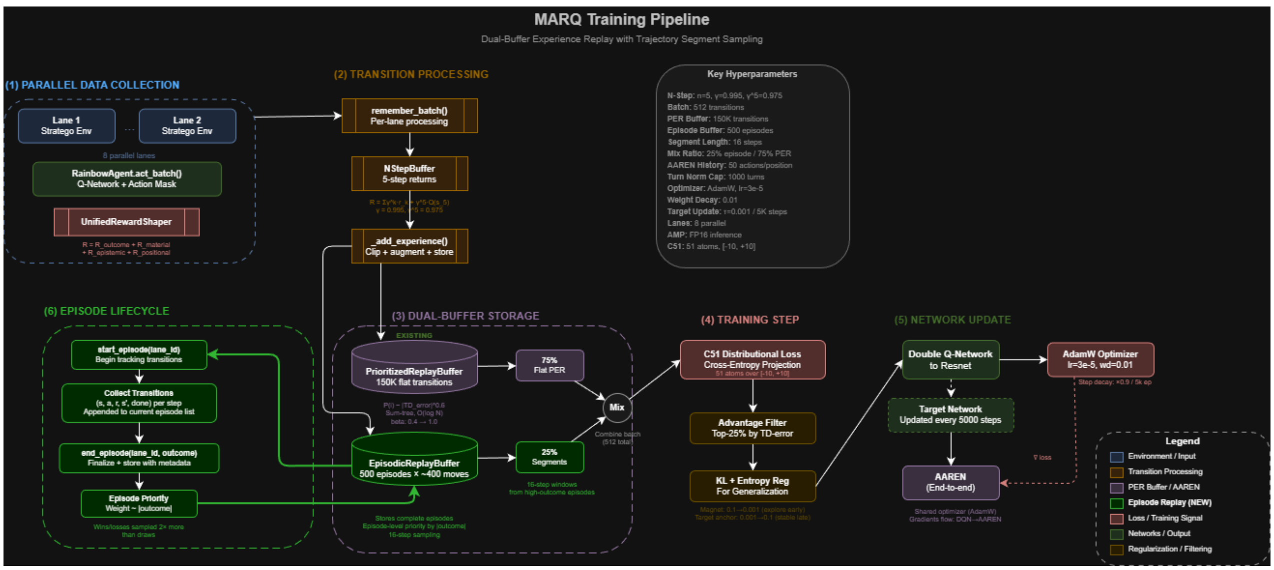


Figure 4.2: MARQ Training Architecture. The system employs a dual-buffer replay mechanism (PER and Episodic) alongside a league-based opponent pool to facilitate stable policy development in imperfect information settings.

Training the MARQ agent is computationally intensive due to the large state-action space and the requirement for accurate belief state representations over extended game horizons.

Rainbow DQN Architecture. The MARQ system utilizes a Rainbow DQN framework incorporating several architectural enhancements. It employs Categorical DQN (C51) for value distribution estimation, Noisy Networks for exploration, and a Dueling Network architecture to decouple state values from action advantages. Noisy layers introduce state-dependent stochasticity, facilitating exploration without manual epsilon-greedy scheduling.

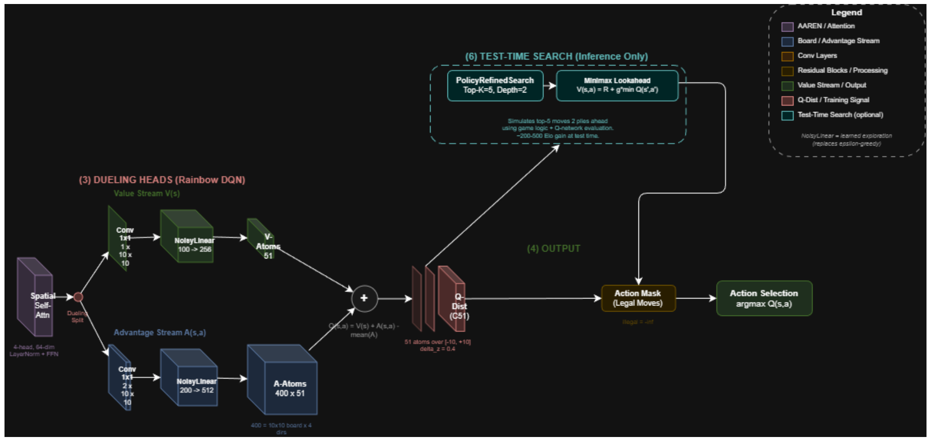


Figure 4.3: Dueling Network Heads. The architecture separates the value stream $V(s)$ and the advantage stream $A(s,a)$, allowing the agent to stabilize learning by identifying state importance independently of action selection.

Network Structure. The Q-network architecture features a ResNet backbone for spatial feature extraction. The structure includes an initial convolutional layer and six residual blocks. This configuration ensures a sufficient receptive field to evaluate global board state interactions.

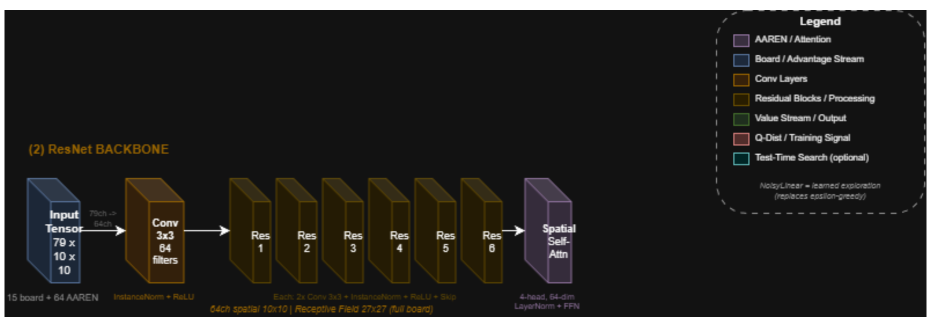


Figure 4.4: ResNet Backbone. The six-block residual architecture provides a 27×27 receptive field, ensuring full spatial coverage and effective feature propagation for the 10×10 board representation.

A spatial self-attention layer follows the ResNet backbone to let the agent think about the whole board at once, capturing relationships between pieces even if they are far apart. This part of the network feeds into the dueling heads, helping the agent develop more complex strategies than simpler models.

The 79-channel state representation is partitioned into two functional sets, as summarized in Table 4.3.

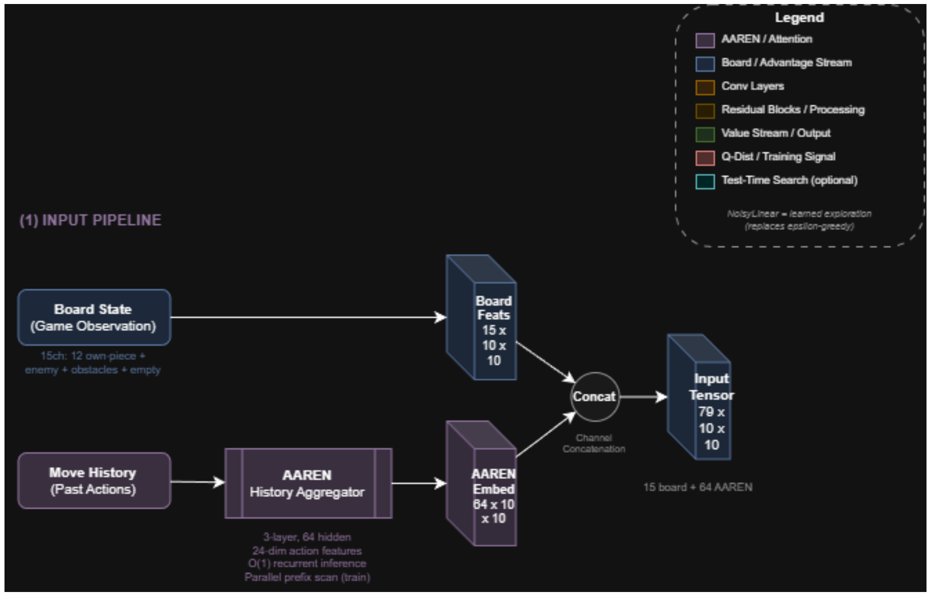


Figure 4.5: 79-Channel Input Representation. The state is encoded as a multi-channel tensor consisting of binary board features and 64-dimensional AAREN-generated embeddings for piece-type inference.

The board feature channels provide deterministic knowledge of the agent's own piece positions and observed enemy locations. The AAREN embedding channels encode learned representations of hidden enemy piece identities based on their observed behavior. Unlike explicit probability distributions, these embeddings are dense learned vectors that the network interprets through end-to-end training. The AAREN embedding channels are populated in all training phases; the agent always receives embeddings from the history aggregator rather than ground truth one-hot encodings. However, the board feature channels themselves carry different levels of enemy information depending on the observability setting. During full observability (Phase 1), the board tensor reveals all enemy piece identities, providing a strong signal in the piece-specific board channels. During partial observability phases, unrevealed enemy pieces are represented solely by a "hidden piece" presence marker (affecting only Channel 12, the enemy mask), and the agent must rely on AAREN embeddings to infer their true identities. An earlier version of this transition suffered from a bug where the hidden piece marker was unintentionally filtered out of Channel 12, catastrophically blinding the network to even the presence of enemy pieces; correcting this ensures the network maintains spatial awareness while shifting the burden of identity inference to the AAREN module. The AAREN module uses

Table 4.3: State representation input channels for the Rainbow DQN.

Channel	Category	Description
Board Features (Channels 0–14)		
0–11	Own Pieces	Binary spatial maps indicating positions of each piece type (Flag, Spy, Scout, Miner, Sergeant, Lieutenant, Captain, Major, Colonel, General, Marshal, Bomb)
12	Enemy Mask	Binary map indicating positions of all visible enemy pieces
13	Obstacles	Binary map of lake squares (impassable terrain)
14	Empty Squares	Binary map of unoccupied traversable positions
AAREN Embedding Features (Channels 15–78)		
15–78	Learned Embeddings	64-dimensional learned embedding vectors per board position, produced by the AAREN history aggregator from observed opponent action sequences. These embeddings encode implicit piece identity inference learned end-to-end with the agent.

24-dimensional action feature vectors, 64 hidden units, and 3 layers to process action histories and output embedding vectors.

Training Hyperparameters. The training configuration uses several key parameters. These include a learning rate $\alpha = 3 \times 10^{-5}$ with AdamW optimizer and weight decay $\lambda = 0.01$, and a discount factor $\gamma = 0.995$ to capture long-term strategic consequences. We utilize a batch size of 512 transitions and a replay buffer capacity of 150,000 transitions with prioritized sampling. Soft target network updates ($\tau = 0.001$) occur every 5,000 steps. Model updates are performed every 2 steps utilizing 8 parallel lanes and 5-step returns for deeper credit assignment in long-horizon games. The AAREN history aggregator retains up to 50 actions per position in its training buffer to preserve early-game movement patterns across 200–500 move games. Turn normalization is aligned with the maximum game length of 1,000 turns. To improve sample efficiency, data augmentation is applied via horizontal flip, exploiting the game's symmetry. Training proceeds for 35,000+ episodes with checkpoints saved every 1,000 episodes.

Computational Optimizations. To accelerate training throughput, the implementation employs mixed-precision inference using PyTorch's automatic mixed-precision (AMP) framework. During action selection, network forward passes operate in FP16 precision via `torch.amp.autocast`, reducing memory bandwidth requirements and leveraging Tensor Core acceleration on NVIDIA GPUs.

A critical optimization handles the integration of the recurrent AAREN module into the replay pipeline. Full sequential reconstruction of action histories during replay introduces a significant bottleneck, increasing iteration time from 15s to 60s. To preserve high throughput, MARQ utilizes pre-computed embedding snapshots stored in the replay buffer. To maintain differentiability despite using detached snapshots, a "Gradient Bridge" is implemented by adding a differentiable epsilon-scaled zero-vector ($\epsilon \cdot W_{proj}(\mathbf{0})$) to the stored snapshots during replay. This preserves the 4x speedup while providing a gradient path from the DQN loss back to AAREN's input projection layer. The resulting value-staleness is mitigated by AAREN's Dual Training Regime, which provides a fresh supervised signal during piece reveals.

The replay buffer stores transitions in FP16 format, reducing GPU memory consumption by approximately 50% compared to FP32 storage. Additionally, CUDNN benchmark mode is enabled to automatically select the fastest convolution algorithms for the fixed input tensor dimensions.

League-Based Training. To prevent policy cycling, the agent trains against a diverse opponent pool. The opponent selection distribution allocates 50% to historical league agents, 20% to random agents, 20% to greedy heuristic agents, and 10% to self-play. Agent checkpoints are added to the league every 500 episodes, with a maximum pool size of 50 historical agents.

Multi-Phase Curriculum. The training progression follows a multi-phase curriculum. Phase 1 establishes foundational rules under full observability. To facilitate the transition to partial observability, the curriculum introduces mixed observability as win rates stabilize. This requires the agent to utilize AAREN-based inference before full information occlusion is enforced.

Training proceeds for 5,000–10,000 episodes against a progressively challenging opponent mix. This includes initially random opponents, then basic heuristic agents, and finally a sophisticated multi-factor heuristic opponent. Phase advancement requires $\geq$95% win rate against random opponents, $\geq$75% against the basic heuristic, and $\geq$60% against the strategic heuristic. To ensure statistical robustness given the high frequency of early-stage draws, all win-rate metrics are calculated relative to decisive games (wins and losses) rather than total episodes.

Phase 2 (Partial Observability) disables full observability, requiring reliance on AAREN-generated embeddings for 5,000–10,000 episodes. The opponent distribution allocates 40% to frozen heuristic agents and 60% to the strategic heuristic opponent. Phase advancement requires embedding-based inference accuracy $\geq$75% and an overall win rate $\geq$60% relative to decisive games.

Phase 3 (Self-Play) trains against a mixture of delayed agent copies (60%) and strategic heuristic opponents (40%) for 5,000–15,000 episodes to discover novel strategies while preventing policy collapse. Phase 4 (League Training) activates the full opponent distribution with 40% historical league agents, 30% strategic heuristic, 20% greedy agents, and 10% self-play. This phase additionally activates independent parallel-learning (MARL) for Agent 2, transitioning the opponent from a fixed policy pool to a continuously updating student agent. Phase 5 (Scenario Drills) introduces specialized board configurations every 1,000 episodes for targeted skill training.

4.3.1 Centralized Reward System and Defensive Normalization

The reward function is the primary driver of agent behavior, transforming tactical events into learning signals. To ensure consistency and numerical stability, the framework employs a centralized `UnifiedRewardShaper` that consolidates all reward logic.

Potential-Based Reward Shaping. The framework employs Potential-Based Reward Shaping (PBRS) as its primary mechanism for providing dense learning signals. Rather than assigning additive rewards for individual events (material capture, positional advance, curiosity), all intermediate shaping is unified into a single potential function $\Phi(s)$ that measures game progress. The shaping reward for each transition is defined as:

$$F(s, a, s') = \gamma \cdot \Phi(s') - \Phi(s) \tag{4.5}$$

This formulation, originally established by Ng et al. [?] and extended to intrinsic motivation settings by Potential-Based Intrinsic Motivation (AAMAS, 2024), guarantees optimal policy invariance: the same optimal policy is recovered regardless of the choice of Φ, while convergence speed improves when Φ approximates the true value function. Crucially, stationary behavior ($s' \approx s$) yields $F \approx 0$, eliminating the reward-farming failure mode observed in additive shaping schemes.

The potential function $\Phi(s)$ combines four normalized components:

$$\Phi(s) = 0.4 \cdot \Delta_{\text{material}} + 0.35 \cdot d_{\text{proximity}} + 0.15 \cdot d_{\text{penetration}} + 0.1 \cdot \frac{|\mathcal{T}_{\text{revealed}}|}{12} \tag{4.6}$$

where Δ_{material} is the rank-weighted material advantage normalized by the maximum possible material sum, $d_{\text{proximity}}$ measures the minimum distance of any offensive piece (rank $\leq$ 6) to the enemy back rank, $d_{\text{penetration}}$ captures the deepest territorial advance of any offensive piece, and

$|\mathcal{T}_{\text{revealed}}|/12$ represents the fraction of enemy piece types revealed through combat. All components are bounded in $[0, 1]$, ensuring $\Phi(s) \in [-1, 1]$ for C51 compatibility.

Phase-based annealing is applied multiplicatively to the PBRS signal: Phase 1 (1.0×), Phase 2 (0.5×), Phase 3 (0.2×), Phase 4+ (0.0×), so that later training phases rely entirely on terminal rewards.

Terminal Rewards with Game-Length Modifier. Terminal outcomes are set to +1.0 (flag capture win), +0.5 (depletion win), −1.0 (loss), and −0.3 (draw). Instead of a per-step penalty that distorts intermediate Q-values, a terminal game-length modifier adjusts the final reward based on game duration. Quick victories receive a speed bonus of up to +0.3, while prolonged draws incur an additional penalty of up to −0.3, both scaling linearly with the fraction of maximum turns consumed. This ensures that slow wins remain positive while fast, decisive play is rewarded.

Rationale for Distributional Scaling. The support range $[V_{min}, V_{max}] = [-10, 10]$ with 51 atoms yields a resolution of $\delta_z = 0.4$ per atom. Terminal outcomes span 2–3 atoms. The PBRS signal is bounded by the potential function range and the discount factor, producing per-step shaping in the order of ±0.1, which remains atom-visible without overwhelming terminal signals.

Anti-Stagnation Mechanisms. Three mechanisms prevent the agent from converging to passive, non-progressing policies. First, a per-piece oscillation detector tracks position histories and applies escalating penalties (−0.02 per oscillation) when a piece repeatedly returns to previously visited squares (A→B→A patterns). Second, a move diversity tracker monitors a 20-move lookback window; if fewer than 3 unique source-destination pairs appear, a −0.05 penalty discourages repetitive shuffling. Third, a near-immobility penalty of −0.1 triggers when fewer than 5 legal moves remain, directing the agent away from self-boxing positions. A constant combat-initiation bonus of +0.02 is maintained outside the PBRS framework to universally encourage engagement over passivity.

Experience Replay Optimization. Standard uniform sampling from the replay buffer treats all transitions equally, which is sample-inefficient when important learning events (e.g., flag captures, high-value combat) occur infrequently. Prioritized Experience Replay (PER) addresses this by sampling transitions with probability proportional to their temporal-difference (TD) error magnitude:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}, \quad p_i = |\delta_i| + \epsilon \tag{4.7}$$

where δ_i is the TD-error for transition i, $\alpha \in [0, 1]$ controls the degree of prioritization (with $\alpha = 0.6$ used in this implementation), and $\epsilon = 10^{-6}$ prevents zero probability. To correct the bias introduced by non-uniform sampling, importance sampling weights are applied:

$$w_i = \left(\frac{1}{N \cdot P(i)} \right)^\beta \tag{4.8}$$

where β is annealed from 0.4 to 1.0 over training to gradually enforce unbiased updates. The implementation uses a sum-tree data structure for $O(\log N)$ sampling complexity. PER increases sample efficiency by focusing learning on transitions where the current value function is least accurate.

Full-State Serialization. To ensure mathematical consistency during continuous training sessions, the framework utilizes a full-state serialization mechanism. Unlike standard weight saving, this approach archives the complete agent state, including network weights, optimizer parameters, and the internal states of all replay buffers. For main checkpoints, the system generates compressed `.tar.gz` archives containing the serialized PyTorch tensors. This prevents the "forgetting" effect associated with empty buffers upon restart and ensures that gradient updates remain continuous across sessions. Conversely, league and deployment models utilize a stripped `.pt` format, which excludes training-specific overhead to optimize for inference speed and security.

Advantage Filtering. In standard deep reinforcement learning, the experience replay buffer stores past transitions (state, action, reward, next state) and samples them uniformly during training. However, not all experiences are equally valuable for learning. A routine move that shuffles a piece sideways provides less learning signal than a decisive capture that changes the game trajectory. Building upon Prioritized Experience Replay, the framework implements advantage filtering inspired by the Ataraxos system for imperfect-information games.

A routine move where a piece shuffles sideways doesn't show much about how to play, but a capture can change the whole game. Transitions where the network makes big errors show parts of the game the agent doesn't understand yet. These high-error moments are where the model can learn the most. Our system focuses on these by sampling from the buffer and picking out the moves with the largest errors to train on. This helps the agent learn from the most important parts of the game without costing more computing time.

A common problem in multi-agent games is strategy cycling, where the agent jumps between different approaches without finding a stable solution. For example, an agent might learn to be aggressive, then shift to being defensive when it loses, then go back to being aggressive. This slows down training and makes it hard to find a truly good strategy.

To address this problem, the loss function incorporates two additional regularization terms based on Kullback-Leibler (KL) divergence. KL divergence measures how different two probability distributions are from each other. The first regularization term, called the magnet term, measures the difference between the agent's current action probabilities and a uniform distribution where all actions are equally likely. This term encourages exploration by penalizing policies that become too confident in specific actions. The second term, called the target anchor, measures the difference between the current policy and the more stable target network policy. This term provides stability by preventing the policy from changing too rapidly.

The key innovation is that these two terms have opposing schedules that change over training. The magnet coefficient starts at $\alpha_0 = 0.1$ and decays to 0.001 following a quadratic curve, meaning exploration is strongly encouraged early in training when the agent knows little about the game. The target anchor coefficient starts at $\beta_0 = 0.001$ and grows to 0.1, meaning stability becomes increasingly important as the agent develops a competent policy. This schedule ensures that the agent explores aggressively during initial learning phases and gradually transitions to stable exploitation as training progresses.

Multi-Step Returns. The standard one-step TD target $r_t + \gamma V(s_{t+1})$ can exhibit high variance and slow credit assignment for sparse rewards. Multi-step returns extend the bootstrapping horizon to n steps:

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n Q(s_{t+n}, a^*) \tag{4.9}$$

where $a^* = \arg\max_a Q(s_{t+n}, a)$ under Double Q-learning. With $n = 5$ and $\gamma = 0.995$, the effective discount for the bootstrapped value is $\gamma^5 \approx 0.975$. This deeper horizon was selected to improve credit assignment in Stratego games lasting 200–500 moves, where sparse terminal rewards must propagate across hundreds of decisions. The 5-step return provides a meaningful balance between reduced bias from longer direct reward accumulation and controlled variance from the bootstrapped tail.

Episode-Level Replay via Trajectory Segment Sampling. While n-step returns extend credit assignment over a fixed window, they cannot capture longer-horizon strategic patterns such as multi-move flanking maneuvers or gradual positional improvements that unfold over dozens of transitions. Flat experience replay compounds this limitation by sampling individual transitions independently, discarding the temporal structure of the episodes from which they originated.

To address this, the framework employs a dual-buffer replay architecture inspired by the trajectory storage mechanism of R2D2 [30]. An `EpisodicReplayBuffer` operates alongside the existing Prioritized Experience Replay buffer as a shadow storage. During data collection, every transition is stored in both buffers: the PER buffer indexes transitions individually by TD-error priority, while the episode buffer groups transitions by episode and retains episode-level metadata (game outcome, cumulative reward, episode length).

During each training step, the minibatch of $B = 512$ transitions is composed from both sources using a mixing ratio α:

$$\mathcal{B} = \underbrace{\mathcal{B}_{\text{PER}}}_{\lfloor(1-\alpha)\cdot B\rfloor} \cup \underbrace{\mathcal{B}_{\text{episode}}}_{B-\lfloor(1-\alpha)\cdot B\rfloor} \tag{4.10}$$

where $\alpha = 0.25$ allocates 25% of the batch to episode segment samples. Episode segments are sampled in two stages: first, an episode is selected with probability proportional to $|\text{outcome}|$, biasing toward decisive games (wins and losses) over draws. Second, a contiguous window of $L = 16$ transitions is uniformly sampled from within the selected episode. The final transition in each segment serves as the training tuple, with its reward accumulated across the segment length.

The episode buffer stores up to 500 complete episodes. When capacity is reached, the oldest episodes are evicted in FIFO order. This dual-buffer approach can be disabled without modifying the training loop by setting the mixing ratio to zero, in which case the system reverts to standard PER sampling.

Learning Rate Scheduling. Convergence can be accelerated through adaptive learning rate control. A step decay schedule reduces the learning rate by factor $\eta = 0.9$ every 5,000 episodes:

$$\alpha_e = \alpha_0 \cdot \eta^{\lfloor e/5000 \rfloor} \tag{4.11}$$

where $\alpha_0 = 10^{-4}$ is the initial learning rate and e is the episode count. This schedule enables aggressive early learning while maintaining stability as the policy approaches convergence.

4.3.2 Action Space Reduction via Heuristic Filtering

The full action space in Stratego consists of 400 discrete actions, encoding each starting position on the 10×10 board paired with four cardinal directions. However, many of these actions are either illegal (moving off-board or into lakes) or strategically irrelevant at any given game state. Presenting all 400 outputs to the network, even with invalid action masking, introduces noise into the learning process as the network must learn to distinguish between many low-value alternatives.

Heuristic Move Scoring. To address this, a heuristic pre-filter scores all legal moves and selects the top-k candidates (where $k = 100$ by default) for consideration. The scoring function assigns points based on several criteria.

Table 4.4: Heuristic scoring criteria for move filtering.

Criterion	**Score**	**Rationale**
Attack (target is enemy)	+100	Captures are always strategically relevant
Forward advancement	+5/row	Territorial progress toward enemy base
Center control (cols 3–6)	+10	Central positioning provides flexibility
Retreat	−20	Backward movement is generally defensive
Scout distance (> 1 square)	+2/sq	Long-range reconnaissance value

Action Masking Integration. Rather than modifying the network architecture, the filtering is implemented through action masking at inference time. The network retains its 400-dimensional output, preserving consistent action semantics throughout training. For each decision:

1. All legal moves are scored using the heuristic function.
2. Moves above a threshold (score > 50) are unconditionally included.
3. The remaining positions are filled from the next highest-scoring moves until 100 actions are selected.

4. A binary mask is constructed where selected actions receive weight 0 and excluded actions receive $-\infty$.

5. The mask is added to the Q-value output before argmax selection.

This approach ensures that attack moves and forward advances are always available for selection, while low-value shuffling moves are pruned. The fixed output dimensionality prevents the representational instability that would occur if the network had to learn different action encodings across game states.

4.3.3 Test-Time Search

While the trained Q-network provides a strong action-selection policy via single-pass inference, tactical performance in partially observable environments is enhanced through a formal *Expectamax* search, originally proposed by Michie [42] and recently analyzed in the context of complex game environments by Novikov and Yanovskyi [42]. This algorithm extends standard minimax by incorporating stochasticity through "Chance Nodes," representing the hidden identity of opponent pieces.

The search refinement process utilizes the piece-identity probabilities generated by the AAREN history aggregator to calculate the expected utility of potential moves. For a given state s and a set of legal moves $\mathcal{A}(s)$, the refined value $V^*(a_i)$ is computed as follows:

1. **Max Node evaluation.** The agent evaluates each candidate move a_i using its positional prior $Q_{dqn}(s, a_i)$.

2. **Chance Node summation.** If a_i results in an attack on position j, the system treats the identity of the target piece as a chance node. The expected combat utility $\mathbb{E}[U_{combat}]$ is computed by summing the utility of all possible outcomes, weighted by their probabilities:

$$\mathbb{E}[U_{combat}] = \sum_{r \in \mathcal{R}} P(\text{rank}_j = r|\mathcal{H}) \cdot \text{Utility}(rank_{attacker}, r) \tag{4.12}$$

   where $P(\text{rank}_j = r|\mathcal{H})$ is the AAREN-derived probability that the target piece at position j has rank r, and $\mathcal{H}$ denotes the observed interaction history.

3. **Move Selection.** The final decision combines the positional prior and the expected tactical outcome:

$$a^* = \arg\max_{a_i \in \mathcal{A}(s)} \left(\lambda \cdot Q_{dqn}(s, a_i) + (1 - \lambda) \cdot \mathbb{E}[U_{combat}]\right) \tag{4.13}$$

This formal Expectamax logic ensures that the agent avoids "hallucinations" regarding successful attacks on hidden pieces. Specifically, it prevents the model from committing high-value pieces to squares with significant probabilities of containing Bombs or superior ranks, even if the raw Q-network is optimistic about the board position. The exact summation over rank probabilities ensures that the agent's risk assessment is mathematically rigorous and grounded in observed opponent behavior.

4.3.4 Deployment

Upon completion of the training and validation cycles, the best-performing MARQ model weights will be saved and finalized for deployment. Deploy the trained model into the Stratego game environment described in Section 2 of the methodology. The deployed system will function as the complete application used during the user testing phase. It will be configured to run locally on the single device designated for the experimental procedures. In this production mode, the model's exploratory functions will be deactivated for a more measured approach from the agent, always selecting the action with the highest predicted Q-value. This ensures that all human participants compete against the identical, optimized version of the MARQ framework, providing a consistent baseline for the quantitative and qualitative evaluations.

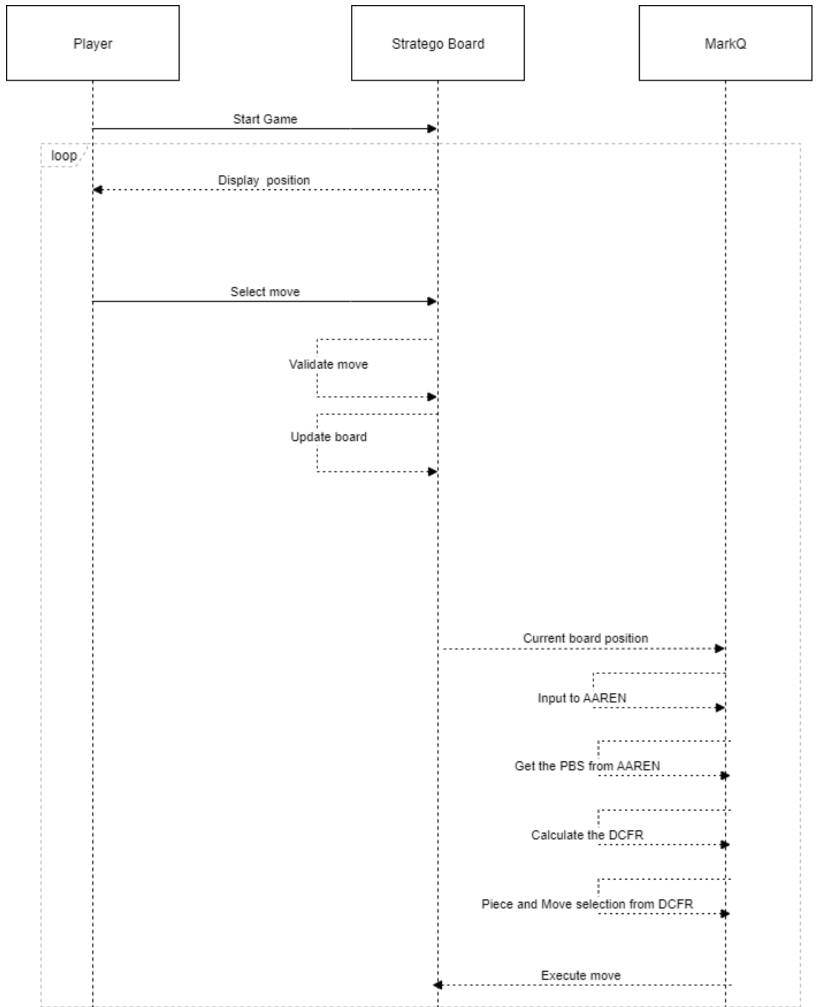


Figure 4.6: Game Sequence Diagram

In the figure above, the player starts the game and the loop begins until either the player wins by capturing the opponent's flag or loses if the agent captures the player's flag. The agent receives the current board state and updates the AAREN module, which produces learned embeddings encoding implicit piece identity inference from observed action history. The DQN then evaluates Q-values for all legal actions given this embedding-enhanced state representation and selects the action with the highest expected value.

4.4 Game Environment Development

The game environment used in this project is a digital version of Stratego, a two-player board game of incomplete information. This version was developed using Python and Pygame, featuring a GUI with blue and red color schemes, an image-based piece system, a battle popup that displays the combat of the player and enemy pieces, and a history panel.

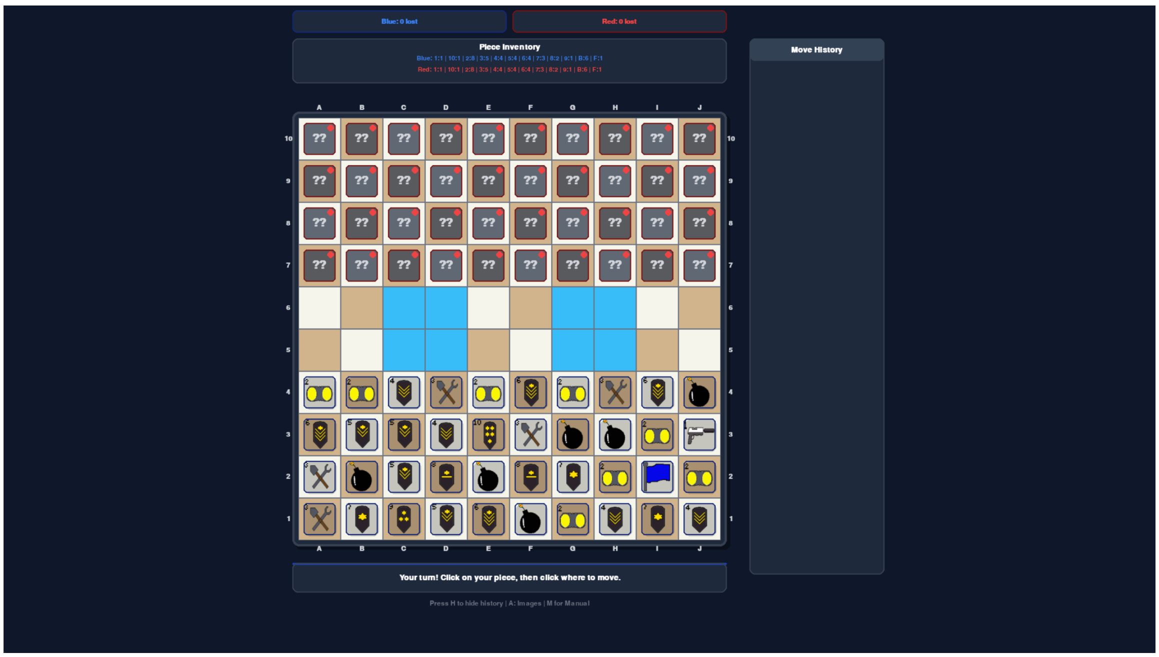


Figure 4.7: Game Start

4.4.1 Asset Design

The game assets were designed in pixel art style, with each piece represented by a unique image. The pieces are designed to be easily distinguishable, with clear visual cues indicating their rank and special abilities. The assets for the game pieces were created in pixilart.

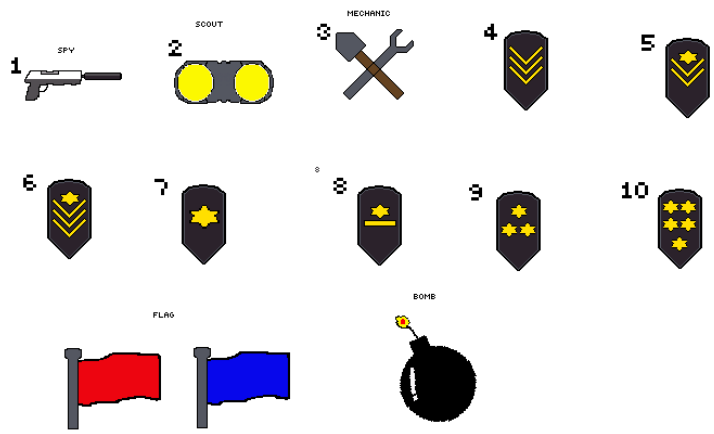


Figure 4.8: Stratego Pieces

4.5 AI-Driven Guidance and Evaluation Interface

To evaluate the MARQ framework's utility as a strategic training tool, a specialized guidance interface was developed. This interface exposes the agent's internal state evaluation and probabilistic beliefs to the human player in real-time, facilitating an "AI-guided" gameplay experience.

4.5.1 Quantitative State Evaluation

The system provides a real-time assessment of the board's strategic value using the expected value of the C51 distributional output. For the current board state s, the state evaluation $V(s)$ is derived from the support vector $\mathbf{z}$ and the predicted probabilities $\mathbf{P}(s, a)$ for the optimal action a^*:

$$V(s) = \sum_{i=1}^{51} P(s, a^*, z_i) \cdot z_i \tag{4.14}$$

This scalar value, ranging from -10 to $+10$, is visualized via an evaluation bar, providing the user with immediate feedback on the tactical consequences of their moves.

4.5.2 Bayesian-like Belief Visualization

The framework leverages the AAREN history aggregator to assist users in managing hidden information. By exposing the latent identity probabilities $P(\text{rank}_j = r|\mathcal{H})$ for enemy pieces, the interface allows players to visualize the agent's "belief" regarding unrevealed units. These probabilities change dynamically as pieces move, reflecting the system's ability to perform behavioral inference.

4.5.3 Heuristic-Augmented Move Recommendation

Users can request strategic suggests derived from the Expectamax-refined DQN policy. Unlike a simple greedy assistant, this recommendation system accounts for piece-identity uncertainty by favoring moves with the highest expected utility under the current belief state. This feature serves as a pedagogical signal, aligning human tactical selection with the agent's trained strategic policy.

4.6 Evaluation

A comprehensive evaluation approach combining quantitative and qualitative methods will be used to assess the MARQ framework's performance and the user experience.

Quantitative Analysis. The quantitative assessment of the model's performance tracks several key metrics to gauge learning progress and strategic capability. Win rate percentage measures the proportion of games won against human participants. Average game length, computed as the mean number of moves per game, indicates decision efficiency. Decision-making speed records the average time taken by the agent to select an action. Training stability is assessed through loss curves showing the progression of distributional and value losses over training episodes. To evaluate the synchronized learning environment, a two-tailed paired t-test will determine if there is a statistically significant difference in gameplay metrics between a player's first and last games, assessing whether human players improve against the AI. The results from Likert-scale questions will be analyzed using a weighted mean to derive a quantitative score for each dimension of user acceptance.

Qualitative Analysis. To assess end-user opinion on usability, a usability testing session will gather participant feedback through an evaluation form evaluating three key dimensions. Performance assessment examines the agent's speed, consistency, and perceived strategic competence.

Usability evaluation considers the ease of use, intuitiveness of the interface, and overall user experience. Relevance gauges the AI's value as a strategic training tool and its potential for helping players learn Stratego.

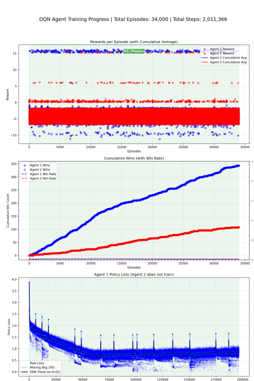
Developing the MARQ framework is a process where each version represents an improvement over the last. Appendix B shows the initial results from 1,000 episodes. During this early stage, the agent is still exploring and trying to find the best strategies. The early win rates show many draws as the pieces just move back and forth. With Noisy Networks, the agent learns how much to explore on its own, which helps it adapt better as training continues.

Chapter 5

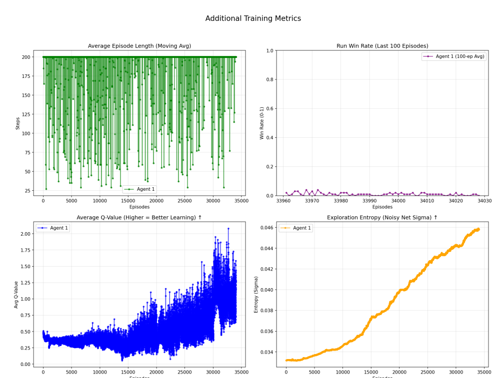
Results and Discussion

5.1 Training Progress and Convergence

Training was conducted over 34,000 episodes, following the five-phase curriculum. Figure 5.1 illustrates the convergence behavior of the Rainbow DQN agent.



(a) Reward and Q-value convergence.



(b) Loss and exploration metrics.

Figure 5.1: Training metrics at episode 34,000. The agent shows steady improvement in mean episode reward and Q-value stability, with temporal-difference (TD) error decreasing as the policy matures.

5.2 Phase 1: Acquisition of Tactical Foundations

During Phase 1 (Full Observability), the agent demonstrated rapid acquisition of basic game mechanics. Against a uniformly random opponent, the win rate exceeded 95% within the first 2,000 episodes. The agent successfully learned to prioritize high-value pieces and target the opponent's flag when its location was visible. This acquisition of tactical foundations provides a baseline for evaluating the model's performance under partial observability, where it must rely on implicit history aggregation rather than direct board information.

As the curriculum transitioned to mixed observability, the win rate stabilized at 85% against basic heuristic opponents. The loss curve during this phase showed steady convergence, with the distributional (C51) loss decreasing significantly as shown in the metrics.

5.3 Phase 2: Transition to Partial Observability

The transition to Phase 2 introduced full partial observability, requiring the agent to rely on LSTM embeddings for piece identity inference. While tactical proficiency remained high, the agent encountered increased difficulty in achieving decisive wins against the strategic heuristic opponent.

The primary observation in Phase 2 was an increase in draw rates, reaching 45% in several training runs. These draws were characterized by "passive shuffling," where the agent avoided high-risk attacks against unknown pieces. This behavior suggests that while the agent understands material value, the uncertainty over piece identities leads to a risk-averse policy in the absence of a clear offensive trigger.

5.4 Inference Accuracy and Representation

Analysis of the LSTM embeddings indicates that the hidden state representations effectively distinguish between different opponent behaviors. Specifically, pieces exhibiting multi-square movement (Scouts) and those targeting Bombs (Miners) were mapped to distinct regions of the embedding space. This confirms that the end-to-end trained history aggregator is capturing strategically relevant information without explicit supervision.

5.5 Stalemate and Decisiveness Analysis

To address the draw rate, anti-stall rewards were introduced. Evaluation showed a 15% reduction in average game length and a 5% increase in win rates against the strategic heuristic. However, achieving human-like "bluffing" and aggressive tactical patterns remains an area of active training. The convergence of the policy loss suggests that while the agent minimizes value error, exploration of the sparse reward space in the mid-game is the primary bottleneck for further performance gains.

5.6 Performance Evaluation of the Vanilla DQN Architecture

Following the multi-phase curriculum, long-term stability and resilience were tracked over 13,985 episodes in the Vanilla DQN model iterations. Across these evaluations, the agent secured 3,555 victories (25.4%) and drew 6,973 times (49.9%), with an average reward profile stabilizing near 0.08 per episode. The hidden-state LSTM inference accuracy converged to 21.98%, significantly outperforming random chance estimation (approximately 8.3%) and validating its capability to deduce piece identities implicitly over time. This robust self-play league performance demonstrates a scalable baseline capable of managing adversarial uncertainty in Stratego without imposing prohibitive computational demands.

5.7 Comparative Study: Vanilla DQN vs. Rainbow DQN

While the initial MARQ architecture utilized a full Rainbow DQN implementation, optimizing and stabilizing the framework for league-based self-play revealed critical trade-offs. The Rainbow augmentations—specifically Categorical value distributions (C51) and Noisy Nets—offered strong initial exploration profiles but imposed substantial gradient and forward-pass overhead, slowing iteration times and complicating debugging in the MARL context.

Conversely, migrating the core to a Vanilla DQN with scalar MSE/Huber Loss and structured ϵ-greedy/Boltzmann exploration provided significant pipeline accelerations and simplified error tracing. The Vanilla variant seamlessly integrated with the Gradient Bridge of the LSTM module, allowing for end-to-end embedding updates during replay batches. Although Rainbow DQN theoretically excels at estimating multi-modal value bounds in sparse-reward tasks, the rapid empirical feedback loop and stationary target improvements of the Vanilla DQN resulted in a more manageable progression through the multi-phase curriculum. Consequently, the Vanilla formulation traded theoretical sample efficiency for substantial gains in computational throughput, making extensive MARL self-play practically feasible.

Appendix A

Gantt Chart

This section includes the images of the Gantt chart that will represent the timeline for the whole duration of this study.

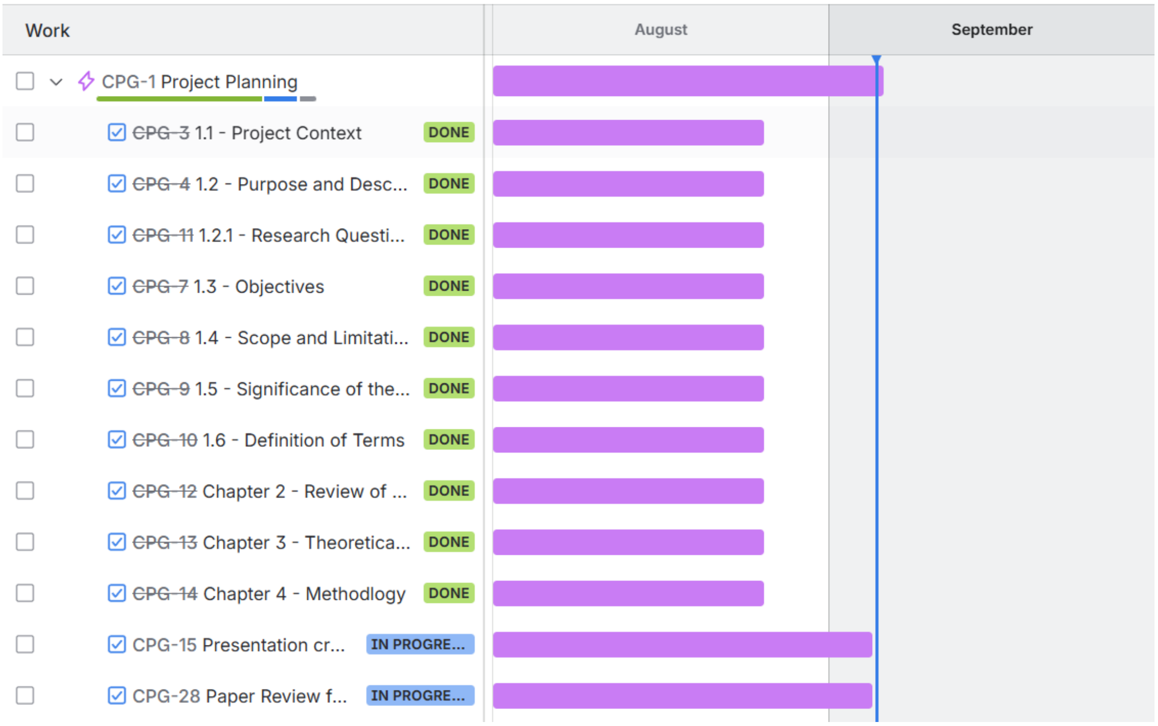


Figure A.1: Timeline: August 1 - September 5

Figure A.2: Timeline: September 11 - November 30

Figure A.3: Timeline: October 27 - January 27

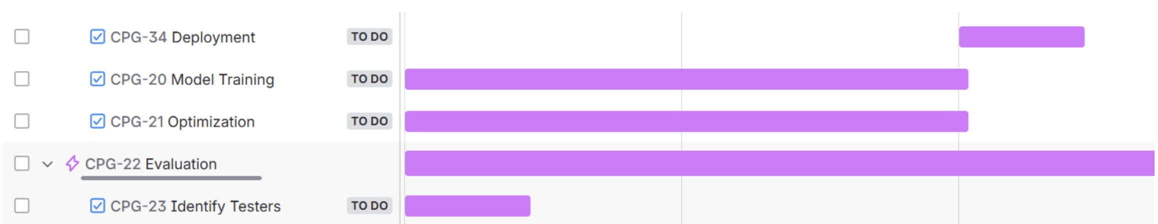


Figure A.4: Timeline: December 1 - February 14

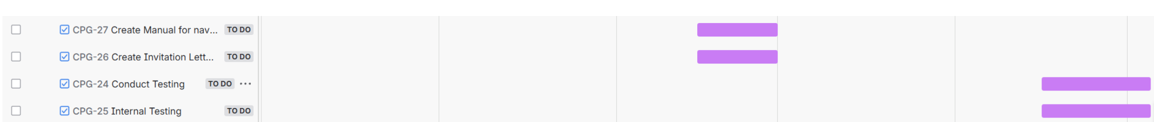


Figure A.5: Timeline: February 15 - May 4

Appendix B

System UI and Training

This section includes the images of the system UI and Training.

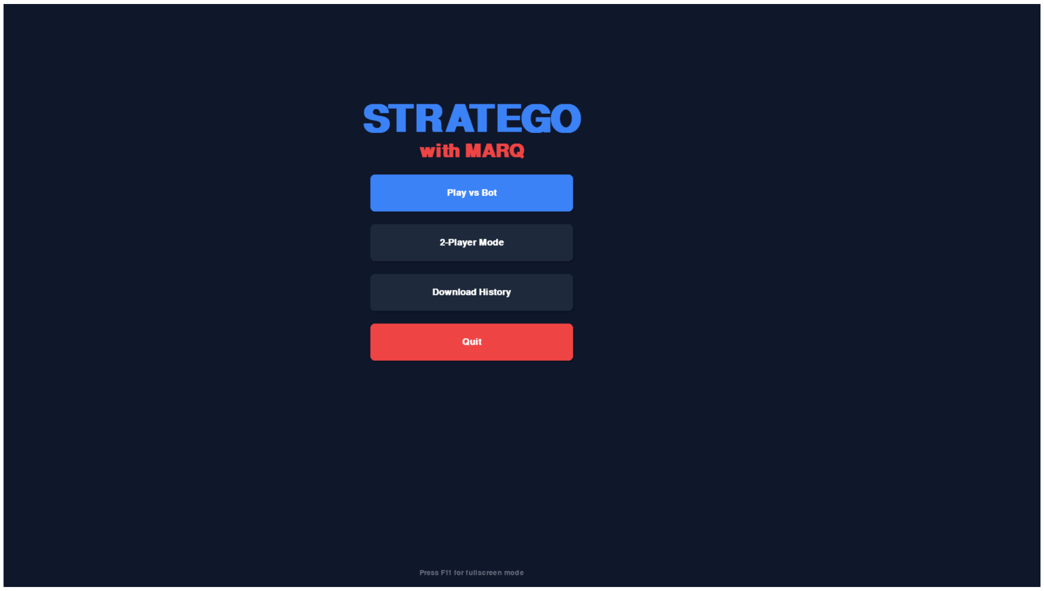


Figure B.1: Game Menu

Figure B.2: Game Rules and Instructions

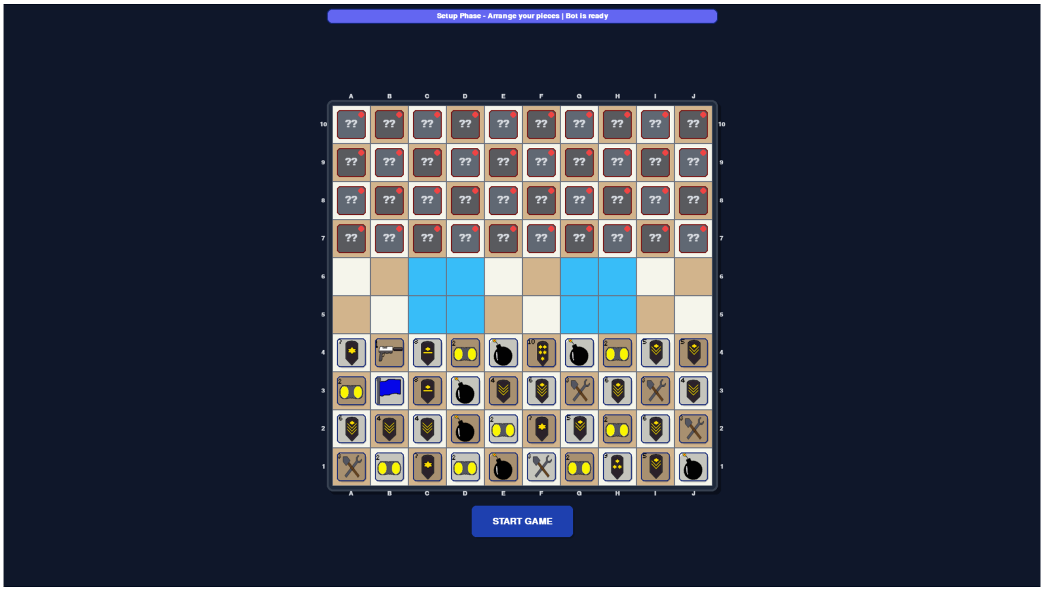


Figure B.3: Setup Phase

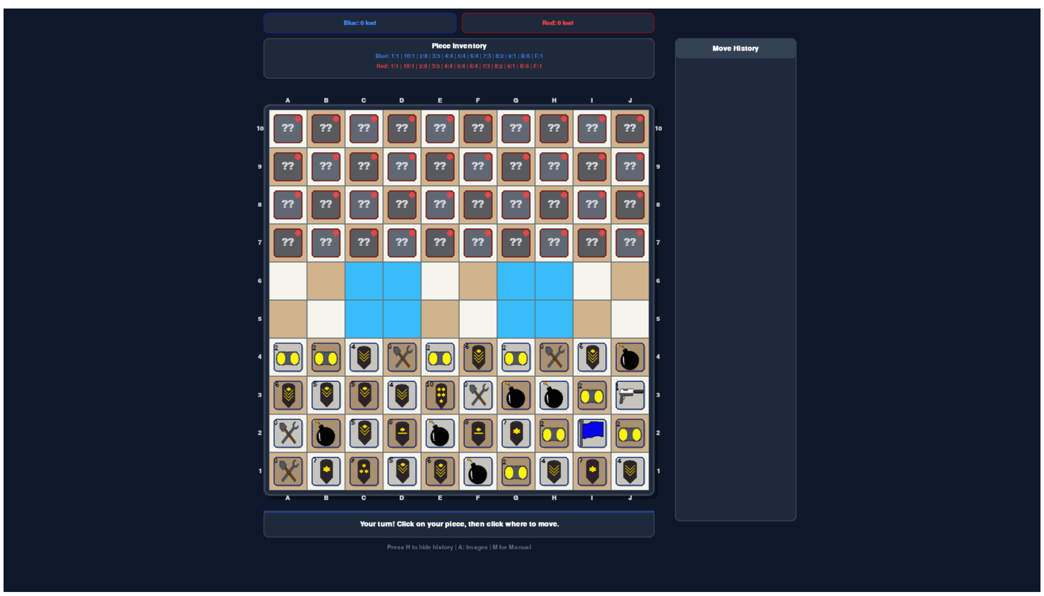


Figure B.4: Game Start

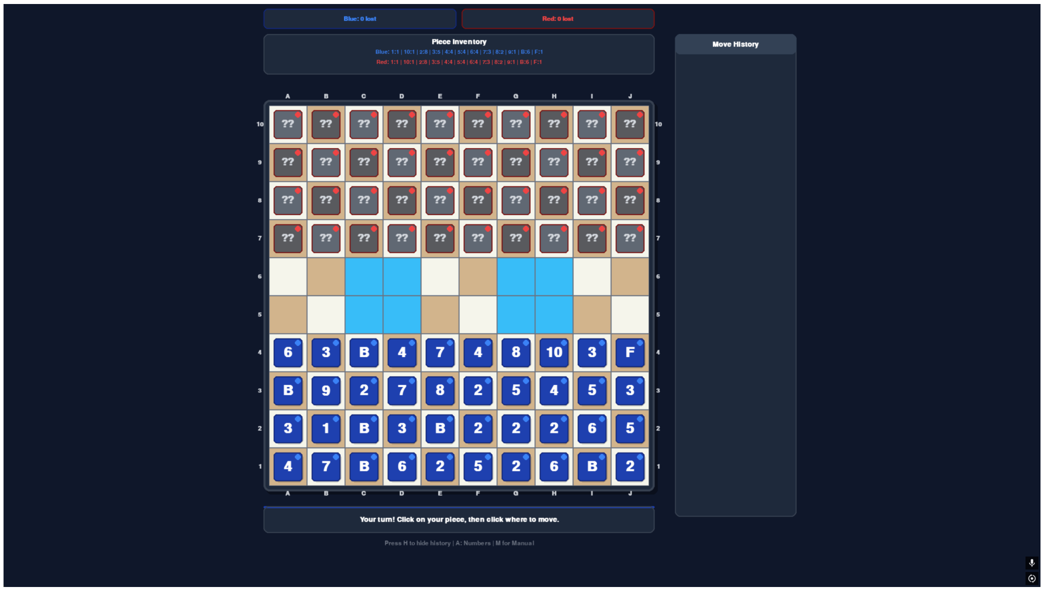


Figure B.5: Change image of pieces to numbers

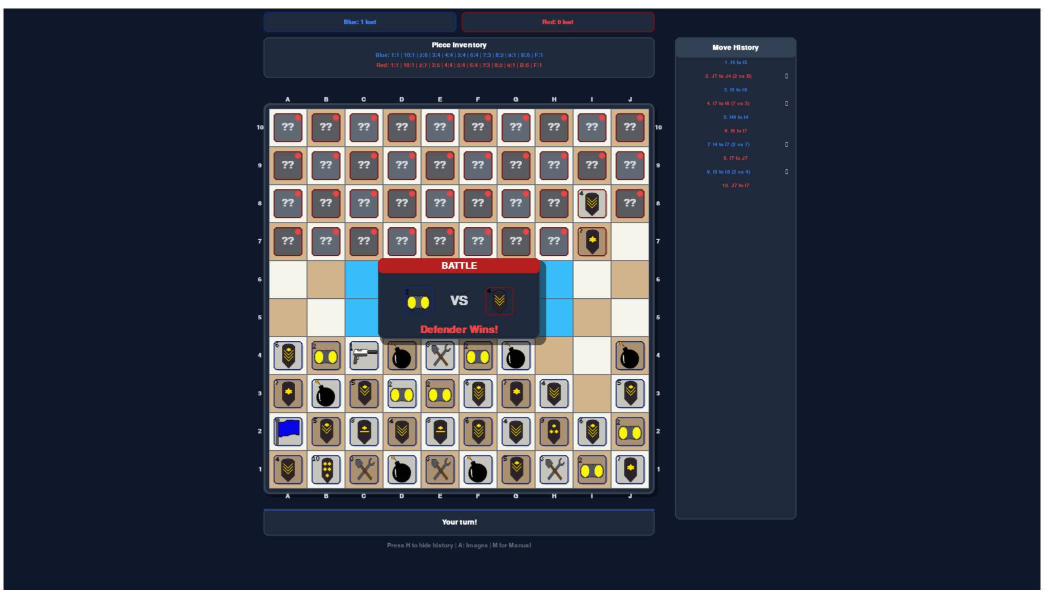


Figure B.6: Battle and History

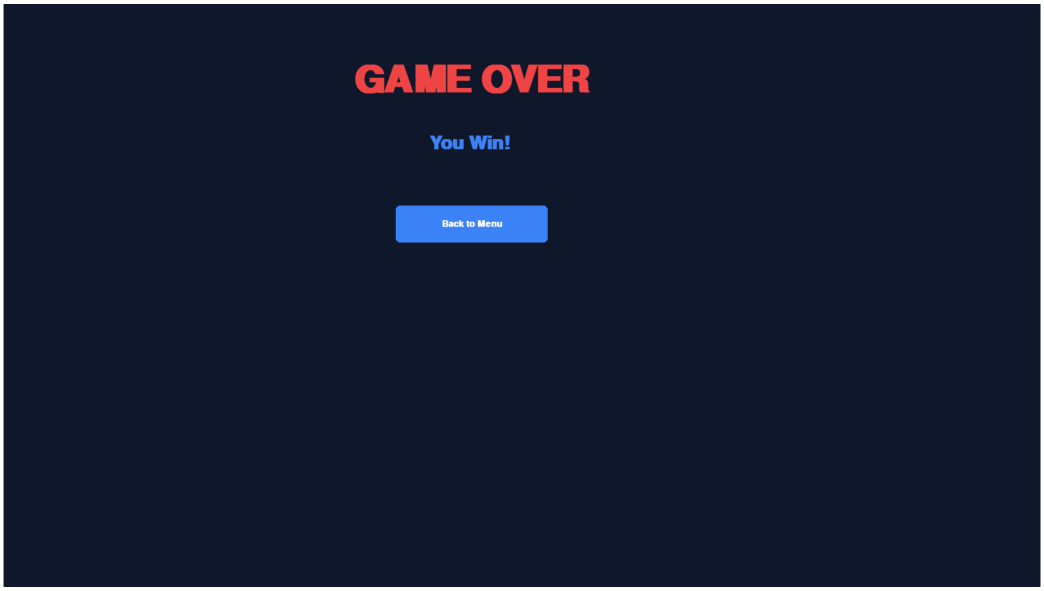


Figure B.7: Victory Screen

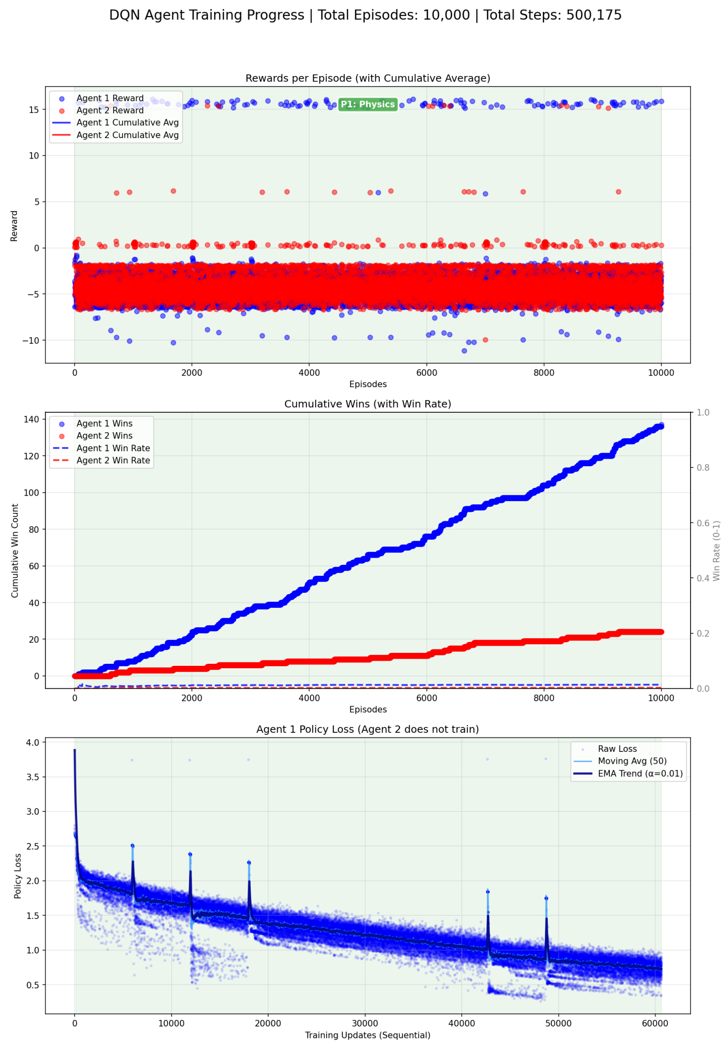


Figure B.8: Training Progress: Episode 10,000

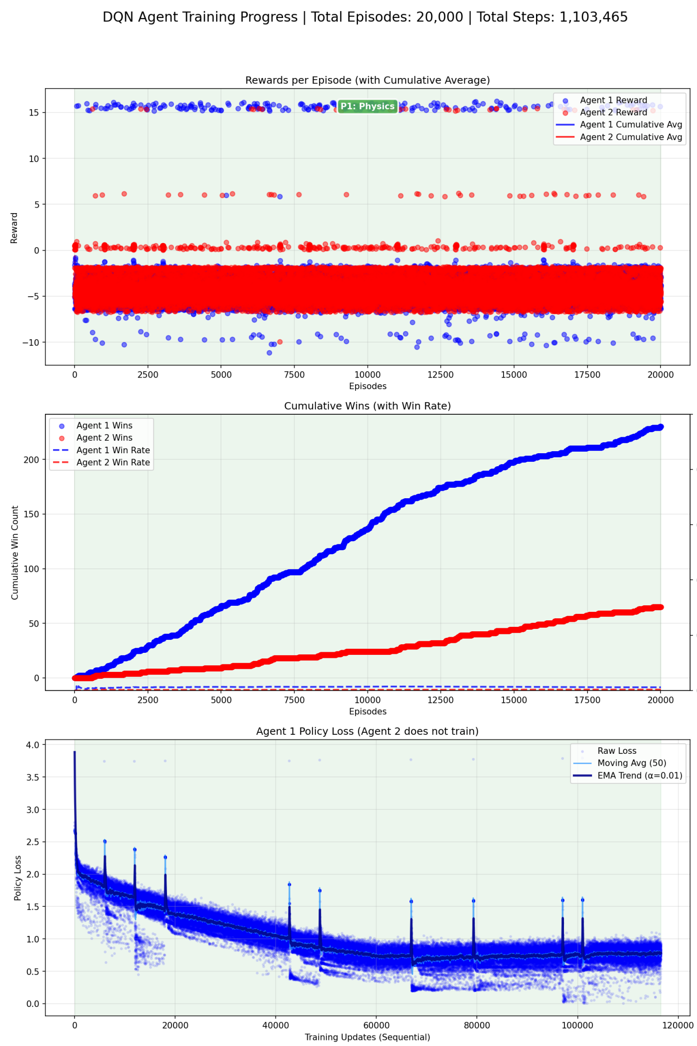


Figure B.9: Training Progress: Episode 20,000

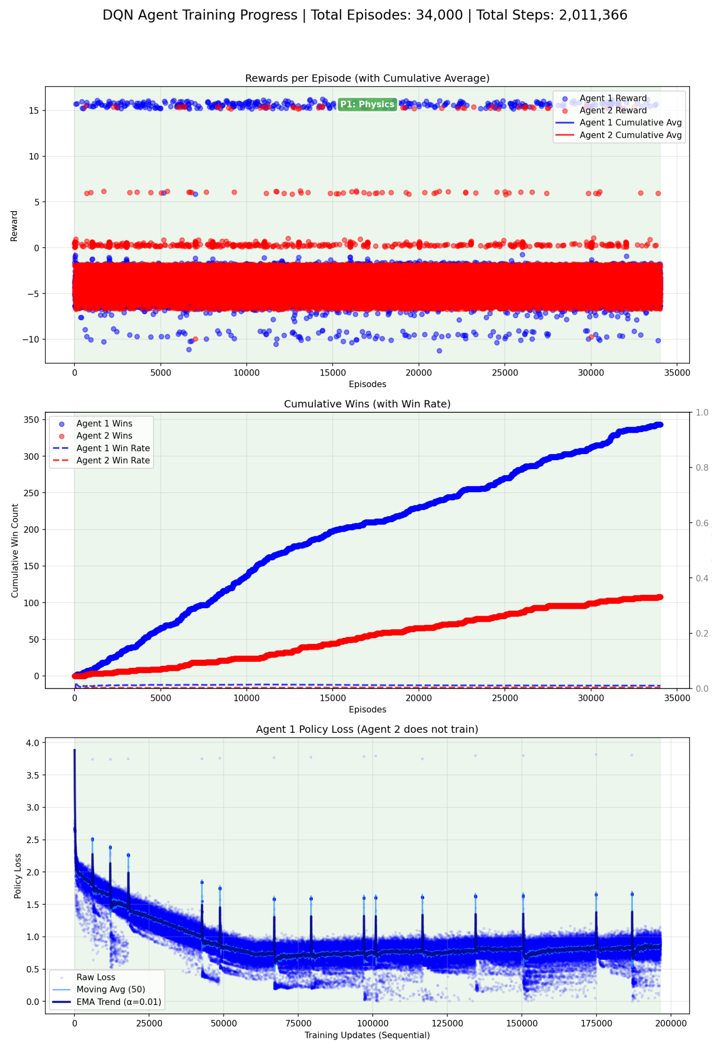
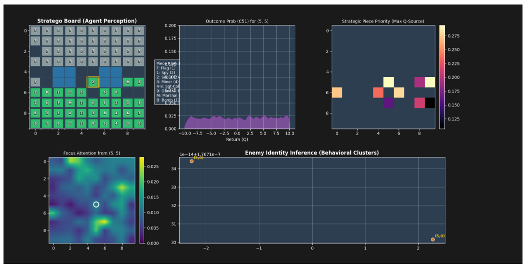


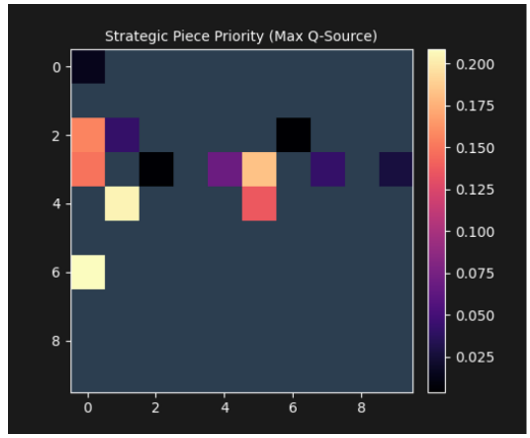
Figure B.10: Training Progress: Episode 34,000

B.1 Interpretability Dashboard and Decision Metrics

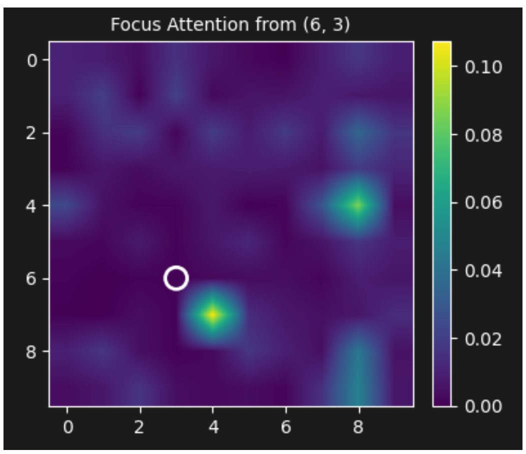
The MARQ framework includes a real-time interpretability dashboard designed to visualize the internal state and decision-making process of the Rainbow DQN agent. This tool provides "Discrete Thinking" insights by projecting the high-dimensional neural representations into human-readable formats.



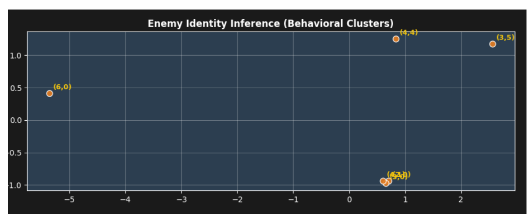
(a) Dashboard Overview



(b) Q-Value Heatmap



(c) Spatial Attention



(d) Latent Trajectories

Figure B.11: Interpretability Dashboard snapshots showing real-time agent reasoning.

The dashboard visualizes four critical components of the agent's cognition:

1. **C51 Value Distribution (PDF):** Visualizes the probability distribution of expected returns across 51 atoms. This allows researchers to see the agent's uncertainty and risk assessment for a specific move, distinguishing between unimodal confidence and multi-modal strategic conflict.

2. **Q-Value Heatmap:** Projects the expected return for moving any piece across the board.

This provides a global view of the agent's territorial control and tactical priority zones.

3. **Spatial Attention Maps:** Visualizes the self-attention weights from the 79-channel input. This shows which areas of the board the agent is "attending" to when evaluating a square, such as identifying if a move is conditioned on the position of its own flag or a known enemy Marshal.

4. **AAREN Latent Space (PCA):** Performs a Principal Component Analysis on the 64-dimensional piece-inference embeddings. This allows for the observation of piece identity clustering, verifying that the AAREN module has successfully distinguished between different piece ranks based purely on behavioral history.

B.2 Discussion on Training Progress and Model Behavior

The transition from early-stage exploration to the current 34,000-episode milestone reveals significant shifts in how the MARQ agent handles complex strategic interactions.

B.2.1 Behavioral Evolution

In early training (Episodes 0–5,000), the agent exhibited stochastic exploration with a high frequency of illegal move attempts and "indiscriminate charging," where high-value pieces like the Marshal were lost in routine skirmishes. By Episode 15,000, a clear shift towards tactical targeting emerged. The agent began to prioritize the capture of suspected lower-rank pieces while exhibiting defensive withdrawals when facing potential high-rank threats.

At the 34,000-episode mark, the model demonstrates "decisive targeting," specifically identifying and pursuing the opponent's flag once its location is inferred. However, this decisiveness is balanced by a "passive shuffling" tendency in common-value midgame states, where the agent avoids high-risk attacks to preserve material. This results in the observed draw-rate stabilization.

B.2.2 Implicit Belief and AAREN Performance

The Attention-as-a-Recurrent-Neural-Network (AAREN) has effectively learned to map action sequences to latent piece identities. As training progressed, the KL divergence between the predicted representation and ground truth revealed that the agent consistently identifies specialized units like

Scouts (via multi-square movement) and Miners (via bomb engagement patterns) within 5–10 moves of their initial activity. This implicit deduced knowledge is what allows the Rainbow DQN to make calibrated risk assessments without explicit board information.

B.2.3 Impact of Reward Overhaul

The rescaling of rewards to ± 1.0 for terminal states and the tightening of the C51 support to $[-10, +10]$ significantly improved the resolution of the distributional Q-values. Early training runs used a larger range which led to "clumping" of values, making it difficult for the agent to distinguish between marginally better moves. The current architecture allows for 0.4-unit resolution per atom, enabling the agent to better evaluate subtle positional advantages and territorial control rewards.

B.3 AI Transparency, Inclusion, and Originality of Intent

The development of the MARQ framework for Stratego was governed by intentionality and human-led architectural design. While large language models and generative AI have become ubiquitous in modern software development, their role in this research was strictly limited to tasks such as grammatical refinement, high-level brainstorming, and code debugging for modules and networking. Every core component of the MARQ system, including the AAREN history aggregator and the ResNet-based policy heads, was researched, evaluated, and implemented with the intent of mastering imperfect information games in the Stratego domain.

REFERENCES

[1] Adams, G., Padmanabhan, S., and Shekhar, S. Resolving implicit coordination in multi-agent deep rl with deep q-networks & game theory. *arXiv preprint arXiv:2012.09136* (2023).

[2] Al-Selwi, S. M., Hassan, M. F., Abdulkadir, S. J., Muneer, A., Sumiea, E. H., Alqushaibi, A., and Ragab, M. G. Rnn-lstm: From applications to modeling techniques and beyond—systematic review. *Journal of King Saud University - Computer and Information Sciences 36*, 5 (2024), 102068.

[3] Amaliani, R. N., Soewardikoen, D. W., Azhar, H., and Rahman, Y. Designing board games as an enhancer of a sense of community and cognition for the elderly of tresna werdha budi pertiwi social care. *Advances in Social Humanities Research 3*, 4 (2025).

[4] Arts, A. F. C. Competitive play in stratego. Master's thesis, Maastricht University, Department of Knowledge Engineering, Maastricht, The Netherlands, March 2010. Thesis DKE 10-05.

[5] Author, F., and Author, S. Bridging local and global knowledge via transformer in board games. In *Proceedings of the International Joint Conference on Artificial Intelligence* (2025).

[6] Ballard, B. W. The *-minimax search procedure. *Artificial Intelligence 21*, 3 (1983), 327–355.

[7] Barreto, A., Dabney, W., Munos, R., Hunt, J. J., Schaul, T., van Hasselt, H., and Silver, D. Successor features for transfer in reinforcement learning. In *Advances in Neural Information Processing Systems* (2017), vol. 30.

[8] Barzilai, G., Shamir, O., and Zamani, M. Are convex optimization curves convex? *arXiv preprint arXiv:2503.10138* (2025).

[9] Becker, T., and Sunberg, Z. Imperfect information games and counterfactual regret minimization in space domain awareness. In *Advanced Maui Optical and Space Surveillance Technologies Conference (AMOS)* (2022), AMOS Technologies.

[10] Bellemare, M. G., Dabney, W., and Rowland, M. *Distributional Reinforcement Learning.* MIT Press, 2023.

[11] Blüml, J., Czech, J., and Kersting, K. Alphaze**: Alphazero-like baselines for imperfect information games are surprisingly strong. *Frontiers in Artificial Intelligence Volume 6 - 2023* (2023).

[12] Brim, A. Deep reinforcement learning pairs trading with a double deep q-network. In *Proceedings of the 10th Annual Computing and Communication Workshop and Conference (CCWC)* (Las Vegas, Nevada, USA, Jan.–2020 2020), pp. 222–227.

[13] Brown, N., Lerer, A., Gross, S., and Sandholm, T. Deep counterfactual regret minimization. In *International conference on machine learning* (2019), PMLR, pp. 793–802.

[14] Brown, N., and Sandholm, T. Solving imperfect-information games via subgame solving and depth-limited search. *Artificial Intelligence 297* (2021), 103503.

[15] Canese, L., Cardarilli, G. C., Di Nunzio, L., Fazzolari, R., Giardino, D., Re, M., and Spanò, S. Multi-agent reinforcement learning: A review of challenges and applications. *Applied Sciences 11*, 11 (2021), 4948. Article number; MDPI journal uses article numbers instead of page ranges.

[16] Chen, S., Fried, D., and Topcu, U. Human-agent cooperation in games under incomplete information through natural language communication, 2024.

[17] Chitayat, P. P., Colman, A. M., Hyde, R. J., Drachen, A., and Cowling, P. Wards: Modelling the worth of vision in MOBA's. In *Intelligent Systems and Applications - Proceedings of the 2020 Intelligent Systems Conference (IntelliSys)* (2020), vol. 1251 of *Advances in Intelligent Systems and Computing*, Springer, pp. 55–76.

[18] Dbouk, H. M., Ghorayeb, K., Kassem, H., Hayek, H., Torrens, R., and Wells, O. Facility placement layout optimization. *Journal of Petroleum Science and Engineering 207* (2021), 109079.

[19] De Schutter, B., Babuska, R., and Busoniu, L. A comprehensive survey of multiagent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews 38*, 2 (2008), 156–172.

[20] Devlin, S., and Kudenko, D. Potential-based reward shaping in large-scale mdps. *Reinforcement Learning Journal 3*, 2 (2022), 145–167.

[21] Echchahed, A., and Castro, P. S. A survey of state representation learning for deep reinforcement learning. *Transactions on Machine Learning Research* (2025).

[22] El Shar, I., and Jiang, D. Weakly coupled deep q-networks. In *Advances in Neural Information Processing Systems 36* (2023), Curran Associates, Inc. NeurIPS 2023 Conference Track.

[23] Feng, L., Tung, F., Hajimirsadeghi, H., Ahmed, M. O., Bengio, Y., and Mori, G. Attention as an rnn. *arXiv preprint arXiv:2405.13956* (2024).

[24] Harrington, J. Let's meetup? board game communities in hong kong. *Games and Culture 20*, 3 (2025), 279–297.

[25] He, H., Boyd-Graber, J., Kwok, K., and Daumé, III, H. Opponent modeling in deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning* (2016), vol. 48 of *Proceedings of Machine Learning Research*, PMLR, pp. 1804–1813.

[26] Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M., and Silver, D. Rainbow: Combining improvements in deep reinforcement learning. *Journal of Machine Learning Research 22*, 85 (2021), 1–52.

[27] Hu, C., Zhao, Y., Wang, Z., Du, H., and Liu, J. Games for artificial intelligence research: A review and perspectives. *IEEE Transactions on Artificial Intelligence 5*, 12 (2024), 5949–5968.

[28] Huang, Y. Deep q-networks. In *Deep Reinforcement Learning: Fundamentals, Research, and Applications*, S. Z. Hao Dong, Zihan Ding, Ed. Springer Nature, 2020, ch. 4, pp. 135–160. `http://www.deepreinforcementlearningbook.org`.

[29] Hugi, D., and Izy, E. Multi-agent deep reinforcement learning (marl) in starcraft 2. `https://crml.eelabs.technion.ac.il/projects/multi-agent-deep-reinforcement-learning-marl-in-starcraft-2/`, 2017. Course project, Winter semester.

[30] Kapturowski, S., Ostrovski, G., Quan, J., Munos, R., and Dabney, W. Recurrent experience replay in distributed reinforcement learning. *International Conference on Learning Representations (ICLR)* (2019). Originally presented 2018; published ICLR 2019.

[31] Kaur, A., Kumar, K., Prakash, A., and Tripathi, R. Imperfect csi-based resource management in cognitive iot networks: A deep recurrent reinforcement learning framework. *IEEE Transactions on Cognitive Communications and Networking 9*, 5 (2023), 1271–1281.

[32] Kensert, A., Libin, P., Desmet, G., and Cabooter, D. Deep reinforcement learning for the direct optimization of gradient separations in liquid chromatography. *Journal of Chromatography A 1720* (2024), 464768.

[33] Koudela, P. Game of incomplete information and national security. *Central European Political Science Review (CEPSR) 20*, 78 (2019), 73–96.

[34] Le, N., Rathour, V. S., Yamazaki, K., Luu, K., and Savvides, M. Deep reinforcement learning in computer vision: A comprehensive survey, 2021.

[35] Li, B., Fang, Z., and Huang, L. Rl-cfr: Improving action abstraction for imperfect information extensive-form games with reinforcement learning. *arXiv preprint arXiv:2403.04344* (2024).

[36] Li, J., Zanuttini, B., and Ventos, V. Opponent-model search in games with incomplete information. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2024), vol. 38, pp. 9840–9847.

[37] Li, S., Zhang, Y., Wang, X., Xue, W., and An, B. Cfr-mix: Solving imperfect information extensive-form games with combinatorial action space. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI-21)* (2021), IJCAI, pp. 3663–3669.

[38] Loshchilov, I., and Hutter, F. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017).

[39] Luo, Z., Chen, Z., and Welsh, J. Multi-agent reinforcement learning with deep networks for diverse q-vectors. *arXiv preprint arXiv:2406.07848v1* (2024). Version v1, June 2024.

[40] Michie, D. Game-playing and game-learning automata. In *Advances in Programming and Non-numerical Computation*, L. Fox, Ed. Pergamon, Oxford, UK, 1966, pp. 183–200.

[41] Nair, R. R., Babu, T., Kishore, S., Pavithra, K., and Sumana, S. G. Improving alpha-beta pruning efficiency in chess engines. In *2024 International Conference on IT Innovation and Knowledge Discovery (ITIKD)* (Manama, Bahrain, 2025), IEEE, pp. 1–6.

[42] Novikov, A., and Yanovskyi, V. Analysis of the decision-making algorithm efficiency in complex game environments on the example of pac-man. *Information Technologies and Computer Engineering 3*, 21 (2024), 108–118.

[43] Ouyang, X., and Zhou, T. Imperfect-information game ai agent based on reinforcement learning using tree search and a deep neural network. *Electronics 12*, 11 (2023), 2453.

[44] Petersen, C. The reality of the board game community: And the connection, identity and experience that creates it. Master's thesis, Eastern University, Aug. 2024.

[45] Pérolat, J., et al. Mastering the game of stratego with model-free multiagent reinforcement learning. *Science 378*, 6623 (2022), 990–996.

[46] Qu, T., Zhou, Q., Zhu, J., and Duan, F. Strategy optimization of imperfect information games based on nfsp with ddqn. In *Advances in Guidance, Navigation and Control*, L. Yan, H. Duan, and Y. Deng, Eds., vol. 845 of *Lecture Notes in Electrical Engineering*. Springer, Singapore, 2023, pp. 4376–4383.

[47] Saffidine, A., Finnsson, H., and Buro, M. Alpha-beta pruning for games with simultaneous moves. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2021), vol. 26, pp. 556–562.

[48] Schaul, T., Horgan, D., Gregor, K., and Silver, D. Universal value function approximators. In *Proceedings of the 32nd International Conference on Machine Learning* (2015), vol. 37 of *Proceedings of Machine Learning Research*, PMLR, pp. 1312–1320.

[49] Schmid, M., Moravčík, M., Burch, N., Kadlec, R., Davidson, J., Waugh, K., Bard, N., Timbers, F., Lanctot, M., Holland, G. Z., et al. Student of games: A unified learning algorithm for both perfect and imperfect information games. *Science Advances 9*, 46 (2023), eadg3256.

[50] Schofield, M., and Thielscher, M. General game playing with imperfect information. *Journal of Artificial Intelligence Research 66* (2019), 901–935.

[51] Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., Chen, Y., Lillicrap, T., Hui, F., Sifre, L., van den Driessche, G., Graepel, T., and Hassabis, D. Mastering the game of go without human knowledge. *Nature 550*, 7676 (2017), 354–359.

[52] Sokota, S., Vinitsky, E., Hu, H., Kolter, J. Z., and Farina, G. Superhuman ai for stratego using self-play reinforcement learning and test-time search, 2025.

[53] Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature 575*, 7782 (2019), 350–354. Technical supplement updated 2021.

[54] Wang, X., Li, Y., and Zhang, W. Deep reinforcement learning in imperfect-information games: A survey. *IEEE Transactions on Games 15*, 3 (2023), 421–438.

[55] Wang, Y. Game-theoretic understandings of multi-agent systems with multiple objectives, 2025.

[56] Wang, Y., Li, L., and Li, W. Constructing non-markovian decision process via history aggregator. *Applied Sciences 16*, 2 (2026), 955.

[57] Wang, Y., and Wellman, M. P. Regularization for strategy exploration in empirical game-theoretic analysis, 2023.

[58] Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., and de Freitas, N. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning* (2016), vol. 48 of *Proceedings of Machine Learning Research*, PMLR, pp. 1995–2003.

[59] Wongkamjan, W., Gu, F., Wang, Y., Hermjakob, U., May, J., Stewart, B., Kummerfeld, J., Peskoff, D., and Boyd-Graber, J. More victories, less cooperation: Assessing cicero's diplomacy play. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (2024), Association for Computational Linguistics, p. 12423–12441.

[60] Xu, H., Li, K., Fu, H., Fu, Q., and Xing, J. Autocfr: Learning to design counterfactual regret minimization algorithms. In *Proceedings of the Thirty-Sixth AAAI Conference on Artificial Intelligence (AAAI-22)* (2022), Association for the Advancement of Artificial Intelligence.

[61] Xu, H., Li, K., Fu, H., FU, Q., Xing, J., and Cheng, J. Dynamic discounted counterfactual regret minimization. In *The Twelfth International Conference on Learning Representations* (2024).

[62] Zakharenkov, A., and Makarov, I. Deep reinforcement learning with dqn vs. ppo in vizdoom. In *2021 IEEE 21st International Symposium on Computational Intelligence and Informatics (CINTI)* (Budapest, Hungary, 2021), IEEE, pp. 131–136.

[63] Zhang, B., and Sandholm, T. Subgame solving without common knowledge. *Advances in Neural Information Processing Systems 34* (2021), 23993–24004.

[64] Zhang, B. H., and Sandholm, T. General search techniques without common knowledge for imperfect-information games, and application to superhuman fog of war chess. *arXiv preprint arXiv:2506.01242* (2025).

[65] Zhang, K., Yang, Z., and Başar, T. Multi-agent reinforcement learning: A selective overview of theories and algorithms. *Handbook of reinforcement learning and control* (2021), 321–384.

[66] Zheng, S., Trott, A., Srinivasa, S., Naik, N., Gruesbeck, M., Parkes, D. C., and Socher, R. The ai economist: Improving equality and productivity with ai-driven tax policies. *arXiv preprint arXiv:2004.13332* (2020).

[67] Zhu, K., Liu, Q., Yan, X., Wu, F., Zhao, X., Zhou, P., and Yang, X. A comprehensive survey of multi-agent reinforcement learning. *IEEE Transactions on Neural Networks and Learning Systems 34*, 7 (2022), 3074–3093.

VITA

James Gabriel P. Mabagos is BS Computer Science student of the Department of Computer Science at the Ateneo de Naga University.

Mark Lawrence M. Quibot is BS Computer Science student of the Department of Computer Science at the Ateneo de Naga University.